

**TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ISO 9001:2015

CAO KHẢI MINH

**XÂY DỰNG SẢN THƯƠNG MẠI ĐIỆN TỬ LĨNH
VỰC NÔNG SẢN Ở TRÀ VINH**

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN**

VĨNH LONG, NĂM 2025

**TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ISO 9001:2015

CAO KHẢI MINH

**XÂY DỰNG SÀN THƯƠNG MẠI ĐIỆN TỬ LĨNH
VỰC NÔNG SẢN Ở TRÀ VINH**

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN**

Giảng viên hướng dẫn: **ThS. LÊ MINH TỰ**

Sinh viên thực hiện: **CAO KHẢI MINH**

Mã số sinh viên: **110121145**

Lớp: **DA21TTC**

Khóa: **2021**

VĨNH LONG, NĂM 2025

LỜI MỞ ĐẦU

Trong thời kỳ chuyển đổi số đang diễn ra mạnh mẽ trên toàn cầu, công nghệ thông tin đóng vai trò then chốt trong việc đổi mới mô hình sản xuất và kinh doanh. Trong đó, thương mại điện tử đã trở thành xu hướng tất yếu, giúp kết nối trực tiếp giữa người bán và người mua, tiết kiệm chi phí trung gian, rút ngắn thời gian giao dịch và mở rộng thị trường tiêu thụ hàng hóa.

Tại Việt Nam, nông nghiệp vẫn là ngành kinh tế quan trọng, đặc biệt tại các tỉnh Đồng bằng sông Cửu Long như Trà Vinh - nơi có thế mạnh về sản xuất nông sản như lúa gạo, trái cây, rau củ. Tuy nhiên, phần lớn nông sản tại đây vẫn được tiêu thụ qua các kênh truyền thống như chợ đầu mối hoặc thương lái, khiến nông dân dễ bị ép giá và thiếu sự chủ động trong việc tiếp cận thị trường.

Trước thực trạng đó, việc xây dựng một sàn thương mại điện tử chuyên biệt cho nông sản tại Trà Vinh là hướng đi phù hợp với xu thế hiện đại và có tính ứng dụng cao. Giải pháp này không chỉ góp phần nâng cao hiệu quả tiêu thụ nông sản mà còn hỗ trợ người dân địa phương tiếp cận thị trường trực tuyến một cách thuận tiện và minh bạch.

Với đề tài này, em mong muốn vận dụng các kiến thức đã học về lập trình web, cơ sở dữ liệu và thiết kế giao diện để xây dựng một hệ thống thương mại điện tử thân thiện, bảo mật và dễ triển khai thực tế. Đây cũng là cơ hội để em nâng cao kỹ năng làm việc với các công nghệ hiện đại như ReactJS, NestJS và MongoDB, đồng thời rèn luyện tư duy phát triển phần mềm một cách chuyên nghiệp.

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Kỹ thuật và Công nghệ đã tận tình hướng dẫn, truyền đạt kiến thức, kỹ năng chuyên môn và tạo điều kiện thuận lợi để em hoàn thành đồ án khóa luận tốt nghiệp này.

Em xin gửi lời cảm ơn đến thầy Lê Minh Tự người đã hướng dẫn, giúp đỡ và động viên em trong suốt quá trình thực hiện đồ án khóa luận tốt nghiệp.

Cuối cùng, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình, bạn bè và những người đã luôn ủng hộ, động viên em trong suốt quá trình học tập và thực hiện đồ án khóa luận tốt nghiệp này.

Em xin chân thành cảm ơn !

NHẬN XÉT
(Của giảng viên hướng dẫn trong đề án, khoá luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

.....

4. Điểm mới đề tài:

.....

.....

.....

.....

.....

.....

5. Giá trị thực trên đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

.....

.....

.....

.....

.....

.....

8. Đánh giá:

.....

.....

.....

.....

Trà Vinh, ngày tháng năm 20...

Giảng viên hướng dẫn

(Ký & ghi rõ họ tên)

This image shows a full page of primary-ruled paper. It features multiple sets of horizontal dotted lines spaced evenly down the page, providing a guide for handwriting practice. The paper is otherwise blank, with no margins or additional markings.

Giảng viên chấm
(ký và ghi rõ họ tên)

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của cán bộ chấm đồ án, khóa luận)

Họ và tên người nhận xét:
Chức danh: Học vị:
Chuyên ngành:
Cơ quan công tác:
Tên sinh viên:
Tên đề tài đồ án, khóa luận tốt nghiệp:
.....
.....

I. Ý KIẾN NHẬN XÉT

1. Nội dung:
-
.....
.....
.....
.....
.....
.....
.....
.....
2. Điểm mới các kết quả của đồ án, khóa luận:
-
.....
.....
3. Ứng dụng thực tế:
-
.....
.....
.....
.....
.....

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

(Các câu hỏi của giáo viên phản biện)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đề án khóa luận tốt nghiệp)

.....

.....

.....

.....

.....

....., ngày tháng năm 20...

Người nhận xét
(Ký & ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU	1
CHƯƠNG: 1 TỔNG QUAN	15
CHƯƠNG: 2 NGHIÊN CỨU LÝ THUYẾT	17
2.1 Tìm hiểu về NodeJS.....	17
2.1.1. NodeJS.....	17
2.1.2. Ứng dụng của NodeJS	17
2.1.3. Nguyên lý hoạt động của NodeJS	18
2.2 Tìm hiểu về React.....	20
2.2.1. React.....	20
2.2.2. Những đặc điểm nổi bật của React.....	20
2.2.3. Các thành phần quan trọng trong ReactJS	21
2.2.4. Những lợi ích tuyệt vời mà ReactJS mang lại cho lập trình viên	22
2.2.5. Các tính năng nổi bật của ReactJS	22
2.2.6. Cách sử dụng ReactJS trong phát triển web.....	24
2.3 Tìm hiểu về RESTful API	25
2.3.1. RESTful API	25
2.3.2. RESTful hoạt động như thế nào	26
2.3.3. API RESTful mang lại những lợi ích	26
2.4 Tìm hiểu về NestJS	28
2.4.1. NestJS	28
2.4.2. Cấu trúc của NestJS.....	28
2.4.3. Hướng dẫn cài đặt NestJS	29
2.5 Tìm hiểu về TypeScript	30
2.5.1. TypeScript	30
2.5.2. Ưu điểm của Typescript	31
2.5.3. Nhược điểm của Typescript	33
2.6 Tìm hiểu về MongoDB	34
2.6.1. MongoDB.....	34
2.6.2. Ưu điểm của MongoDB	34
2.6.3. Nhược điểm của MongoDB	35
CHƯƠNG: 3 HIỆN THỰC HÓA NGHIÊN CỨU	36

3.1	Mô tả bài toán	36
3.2	Yêu cầu chức năng.....	36
3.3	Thiết kế cơ sở dữ liệu	38
3.4	Mô hình thực thể.....	41
3.5	Biểu đồ flowchart	42
3.5.1.	Biểu đồ luồng khách hàng đặt hàng	42
3.5.2.	Biểu đồ luồng nhà cung cấp xử lý đơn hàng	43
3.5.3.	Biểu đồ luồng quản trị viên duyệt sản phẩm.....	44
3.6	Biểu đồ Use Case.....	45
3.6.1.	Biểu đồ Use Case tổng quát	45
3.6.2.	Biểu đồ Use Case tác nhân người dùng	46
3.6.3.	Biểu đồ Use Case tác nhân nhà cung cấp.....	46
3.6.4.	Biểu đồ Use Case tác nhân quản trị.....	47
3.7	Kiến trúc hệ thống	48
CHƯƠNG: 4 KẾT QUẢ NGHIÊN CỨU		50
4.1	Giao diện người dùng	50
4.1.1.	Giao diện trang chủ	50
4.1.2.	Giao diện đăng nhập.....	50
4.1.3.	Giao diện sản phẩm	51
4.1.4.	Giao diện chi tiết sản phẩm	52
4.1.5.	Giao diện giỏ hàng	52
4.1.6.	Giao diện đơn hàng	53
4.2	Giao diện nhà cung cấp.....	54
4.2.1.	Giao diện đăng ký nhà cung cấp	54
4.2.2.	Giao diện thêm sản phẩm	54
4.2.3.	Giao diện quản lý sản phẩm	55
4.2.4.	Giao diện quản lý đơn hàng	56
4.2.5.	Giao diện thống kê doanh thu	56
4.3	Giao diện quản trị	57
4.3.1.	Giao diện quản lý người dùng	57
4.3.2.	Giao diện duyệt nhà cung cấp	57
4.3.3.	Giao diện quản lý sản phẩm	58

4.3.4.	Giao diện quản lý đơn hàng	58
4.3.5.	Giao diện duyệt sản phẩm	59
4.3.6.	Giao diện đánh giá sản phẩm	59
4.3.7.	Giao diện thống kê doanh thu	60
CHƯƠNG: 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		61
5.1.	Kết luận.....	61
5.2.	Hướng phát triển	62
DANH MỤC TÀI LIỆU THAM KHẢO		63
PHỤ LỤC: CÔNG CỤ VÀ CÔNG NGHỆ.....		64

DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

Hình 2.1 NodeJs	17
Hình 2.2 Ứng dụng của NodeJs.....	17
Hình 2.3 React.....	20
Hình 2.4 REST API.....	25
Hình 2.5 Cách hoạt động của RESTful API	26
Hình 2.6 NestJs.....	28
Hình 2.7 TypeScript.....	31
Hình 2.8 MongoDB	34
Hình 3.1 Mô hình thực thể.....	41
Hình 3.2 Biểu đồ luồng xử lý đặt hàng.....	42
Hình 3.3 Biểu đồ luồng xử lý đơn hàng.....	43
Hình 3.4 Biểu đồ luồng xử lý duyệt sản phẩm.....	44
Hình 3.5 Biểu đồ use case tổng quát	45
Hình 3.6 Biểu đồ use case người dùng.....	46
Hình 3.7 Biểu đồ use case nhà cung cấp.....	46
Hình 3.8 Biểu đồ use quản trị.....	47
Hình 3.9 Kiến trúc hệ thống	48
Hình 4.1 Giao diện trang chủ.....	50
Hình 4.2 Giao diện đăng nhập	50
Hình 4.3 Giao diện sản phẩm.....	51
Hình 4.4 Giao diện chi tiết sản phẩm.....	52
Hình 4.5 Giao diện giỏ hàng	52
Hình 4.6 Giao diện đơn hàng	53
Hình 4.7 Giao diện đăng ký nhà cung cấp	54
Hình 4.8 Giao diện thêm sản phẩm	54
Hình 4.9 Giao diện quản lý sản phẩm	55
Hình 4.10 Giao diện quản lý đơn hàng	56
Hình 4.11 Giao diện thống kê doanh thu	56
Hình 4.12 Giao diện quản lý người dùng.....	57
Hình 4.13 Giao diện duyệt nhà cung cấp.....	57
Hình 4.14 Giao diện quản lý sản phẩm.....	58

<i>Hình 4.15 Giao diện quản lý đơn hàng</i>	<i>58</i>
<i>Hình 4.16 Giao diện duyệt sản phẩm</i>	<i>59</i>
<i>Hình 4.17 Giao diện đánh giá sản phẩm.....</i>	<i>59</i>
<i>Hình 4.18 Giao diện thống kê doanh thu</i>	<i>60</i>

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
API	Application Programming Interface
CSDL	Cơ sở dữ liệu
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JSX	JavaScript XML
JWT	JSON Web Token
RDBMS	Relational Database Management System
REST	REpresentational State Transfer
SPA	Single Page Application
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
XML	Extensible Markup Language

CHƯƠNG: 1 TỔNG QUAN

1. Lý do chọn đề tài

Trong bối cảnh hiện nay, chuyển đổi số trong lĩnh vực nông nghiệp đang trở thành một trong những xu hướng tất yếu, góp phần nâng cao hiệu quả sản xuất, phân phối và tiêu thụ nông sản. Tuy nhiên, thực tế tại nhiều địa phương đặc biệt là tỉnh Trà Vinh cho thấy người nông dân và các doanh nghiệp nông sản vẫn gặp không ít khó khăn trong việc tiếp cận thị trường rộng lớn. Nguyên nhân chủ yếu đến từ việc thiếu nền tảng quảng bá phù hợp, phụ thuộc vào kênh trung gian và hạn chế trong việc ứng dụng công nghệ số vào quy trình tiêu thụ.

Trong khi đó, người tiêu dùng hiện đại có xu hướng tìm kiếm các sản phẩm nông sản sạch, an toàn và rõ nguồn gốc thông qua các kênh mua sắm trực tuyến. Tuy nhiên, việc kết nối giữa nguồn cung địa phương và nhu cầu của người tiêu dùng còn thiếu tính hệ thống, minh bạch và tiện lợi.

Từ thực tiễn đó, việc xây dựng một sàn thương mại điện tử chuyên về lĩnh vực nông sản tại Trà Vinh không chỉ là hướng đi phù hợp với xu thế công nghệ hiện nay mà còn là giải pháp thiết thực nhằm hỗ trợ nông dân địa phương quảng bá, tiêu thụ sản phẩm hiệu quả hơn. Đồng thời, sàn thương mại điện tử này còn giúp người tiêu dùng tiếp cận được các sản phẩm chất lượng với giá cả hợp lý và minh bạch về thông tin xuất xứ.

2. Mục đích nghiên cứu

Trong đồ án khóa luận tốt nghiệp này, em sẽ xây dựng sàn thương mại điện tử lĩnh vực nông sản ở Trà Vinh

3. Đối tượng và phạm vi nghiên cứu

Người tiêu dùng: là những khách hàng có nhu cầu mua nông sản trực tuyến.

Nhà cung cấp: bao gồm nông dân, hợp tác xã, doanh nghiệp sản xuất nông sản tại Trà Vinh.

Quản trị viên: người điều phối và giám sát hoạt động của hệ thống.

Đề tài tập trung xây dựng các chức năng cốt lõi của một sàn thương mại điện tử trong phạm vi lĩnh vực nông sản địa phương.

4. Phương pháp nghiên cứu

- Phương pháp lý thuyết:

Nghiên cứu các khái niệm về thương mại điện tử, kiến trúc hệ thống web, quản trị cơ sở dữ liệu, cùng với các công nghệ phổ biến như: NestJS, ReactJS, MongoDB.

Tìm hiểu các mô hình thương mại điện tử phổ biến như Shopee, Tiki, Lazada để học hỏi cấu trúc chức năng, giao diện người dùng và trải nghiệm sử dụng.

Tham khảo các bài viết, luận văn, tài liệu kỹ thuật và báo cáo nghiên cứu liên quan.

- Phương pháp thực nghiệm:

Phân tích yêu cầu thực tế từ người dùng mục tiêu.

Thiết kế kiến trúc hệ thống theo hướng module hóa, dễ bảo trì và mở rộng.

Xây dựng giao diện người dùng bằng ReactJS, đảm bảo tính trực quan và dễ sử dụng cho mọi đối tượng.

Phát triển backend bằng NestJS, triển khai các API phục vụ nghiệp vụ chính của hệ thống.

Sử dụng MongoDB để lưu trữ dữ liệu về sản phẩm, đơn hàng, người dùng và thống kê.

Tiến hành kiểm thử, đánh giá hiệu năng hệ thống và đề xuất hướng phát triển trong tương lai.

CHƯƠNG: 2 NGHIÊN CỨU LÝ THUYẾT

2.1 Tìm hiểu về NodeJS

2.1.1. NodeJS



Hình 2.1 NodeJs

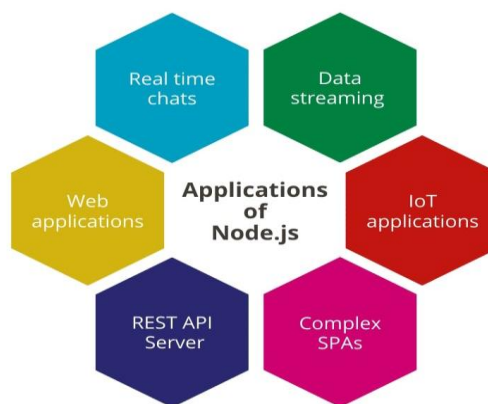
Nodejs là một nền tảng (Platform) phát triển độc lập được xây dựng ở trên Javascript Runtime của Chrome mà chúng ta có thể xây dựng được các ứng dụng mạng một cách nhanh chóng và dễ dàng mở rộng.

Nodejs được xây dựng và phát triển từ năm 2009, bảo trợ bởi công ty Joyent, trụ sở tại California, Hoa Kỳ.

Phần Core bên dưới của Nodejs được viết hầu hết bằng C++ nên cho tốc độ xử lý và hiệu năng khá cao.

Nodejs tạo ra được các ứng dụng có tốc độ xử lý nhanh, realtime thời gian thực [9].

2.1.2. Ứng dụng của NodeJS



Hình 2.2 Ứng dụng của NodeJs

Ứng dụng Web Thời Gian Thực (Real-time Web Applications): Node.js là lựa chọn lý tưởng cho các ứng dụng web thời gian thực như trò chuyện trực tuyến và trò chơi trực tuyến do khả năng xử lý các sự kiện I/O một cách nhanh chóng và hiệu quả.

APIs Server-side: Node.js thường được sử dụng để xây dựng RESTful APIs do khả năng xử lý đồng thời lớn và tốc độ phản hồi nhanh, làm cơ sở cho các ứng dụng di động và web.

Streaming Data: Node.js hỗ trợ xử lý dữ liệu dạng stream, cho phép ứng dụng xử lý các tệp video, âm thanh hoặc các dữ liệu khác trong khi chúng vẫn đang được truyền, thay vì phải chờ cho đến khi toàn bộ tệp được tải về.

Ứng dụng Một Trang (Single Page Applications): Node.js phù hợp với việc phát triển các ứng dụng một trang (SPA) như Gmail, Google Maps, hay Facebook, nơi mà nhiều tương tác xảy ra trên một trang duy nhất mà không cần tải lại trang.

Công cụ và Tự Động Hóa: Node.js cũng được sử dụng để phát triển các công cụ dòng lệnh và các script tự động hóa quy trình làm việc, nhờ vào các gói NPM hỗ trợ đa dạng và khả năng tích hợp dễ dàng với các công nghệ khác.

Microservices Architecture: Node.js là một lựa chọn phổ biến cho kiến trúc microservices, nơi các ứng dụng lớn được chia thành các dịch vụ nhỏ, độc lập và dễ quản lý hơn.

Ứng dụng IoT (Internet of Things): Node.js thường được sử dụng trong các ứng dụng IoT, nơi cần xử lý một lượng lớn các kết nối đồng thời và các sự kiện từ các thiết bị IoT.

Dashboard và Monitoring: Node.js được sử dụng để xây dựng các dashboard hiển thị dữ liệu thời gian thực và các công cụ giám sát, giúp doanh nghiệp dễ dàng theo dõi hiệu suất và tình trạng của các hệ thống [9].

2.1.3. Nguyên lý hoạt động của NodeJS

Node.js là một nền tảng phát triển phía server được xây dựng trên nền tảng JavaScript. Khi một yêu cầu mạng đến từ một client, Node.js sẽ xử lý yêu cầu đó bằng cách thực hiện các bước như sau:

Node.js tạo một event loop để theo dõi các yêu cầu mạng đến và đi.

Khi một yêu cầu mạng đến, Node.js sẽ tạo một worker thread (luồng làm việc) để xử lý yêu cầu đó.

Trong worker thread, Node.js sẽ thực hiện các tác vụ xử lý yêu cầu, chẳng hạn như đọc và ghi vào cơ sở dữ liệu, đọc và ghi file, tương tác với API, ...

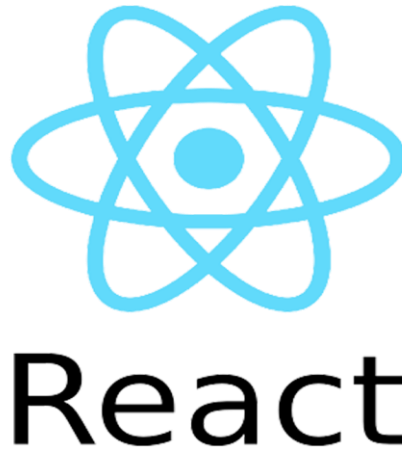
Khi worker thread đã hoàn thành các tác vụ, Node.js sẽ trả về kết quả cho client qua mạng.

Nếu có yêu cầu mạng mới đến, Node.js sẽ tạo một worker thread mới để xử lý yêu cầu đó.

Các yêu cầu mạng đến và đi trong Node.js được xử lý bằng cách sử dụng các hàm callback, Promise, async/await để đảm bảo tính phi đồng bộ và tăng hiệu suất của ứng dụng. Các yêu cầu mạng được xử lý một cách độc lập, giúp tránh tình trạng "blocking" (chặn) trong quá trình xử lý yêu cầu. Node.js cũng có thể hoạt động với các module và thư viện khác để hỗ trợ cho việc phát triển ứng dụng web [4].

2.2 Tìm hiểu về React

2.2.1. React



Hình 2.3 React

React (ReactJS) là một thư viện JavaScript mã nguồn mở, được dùng để xây dựng giao diện người dùng (frontend) cho web. React chỉ tập trung vào phân hiển thị giao diện (view), chứ không can thiệp vào cách sắp xếp logic nghiệp vụ hoặc cấu trúc ứng dụng [1].

2.2.2. Những đặc điểm nổi bật của React

Linh hoạt trong thiết kế kiến trúc

React tập trung vào việc hiển thị giao diện người dùng và cho phép lập trình viên tự do quyết định cách sắp xếp logic nghiệp vụ. Chính sự linh hoạt này làm cho React trở nên phổ biến, vì react phù hợp với nhiều loại dự án và phong cách phát triển khác nhau.

Khác với các framework có kiến trúc cố định như Angular, React không ép buộc người dùng vào một mô hình cụ thể, khiến react linh hoạt cho nhiều dự án khác nhau. Mặc dù điều này trong React đem lại lợi ích cho lập trình viên có kinh nghiệm, nhưng lại gây khó khăn cho người mới bắt đầu vì thiếu sự hướng dẫn cụ thể.

Kiến trúc Component đơn giản và nhẹ

React được xây dựng dựa trên kiến trúc component, nơi mỗi thành phần có thể được tái sử dụng, giúp ứng dụng dễ mở rộng và duy trì. Các component trong React rất nhẹ và có thể chỉ là các hàm đơn giản trả về JSX.

Điều này giúp các lập trình viên dễ dàng phát triển các ứng dụng lớn hơn bằng cách chia nhỏ thành các thành phần độc lập, có thể tái sử dụng mà không cần phải lo về việc tích hợp phức tạp.

Cộng đồng hỗ trợ lớn

Một trong những lý do chính khiến React trở nên phổ biến là nhờ vào số lượng người dùng lớn và sự hỗ trợ từ cộng đồng cũng như các công cụ học tập. Đây là yếu tố quan trọng giúp các lập trình viên, đặc biệt là người mới bắt đầu, dễ dàng học và sử dụng React một cách hiệu quả.

Các diễn đàn lớn như Stack Overflow, Reddit, Dev.to và Github chứa đầy các câu hỏi, câu trả lời và bài viết liên quan đến React, giúp người học dễ dàng tìm thấy sự trợ giúp khi gặp khó khăn.

React được duy trì và phát triển bởi Facebook (Meta), một trong những công ty công nghệ lớn nhất thế giới. Điều này đảm bảo rằng React sẽ nhận được sự hỗ trợ liên tục và các bản cập nhật mới, giúp lập trình viên yên tâm rằng React sẽ tiếp tục phát triển và phù hợp với các dự án dài hạn [1].

2.2.3. Các thành phần quan trọng trong ReactJS

JSX (JavaScript XML) là một cú pháp mở rộng cho phép lập trình viên viết mã giống như HTML trong JavaScript. Trong các ngôn ngữ khác, lập trình viên thường phải viết code HTML và JavaScript riêng rẽ. Tuy nhiên, với JSX, React cho phép lập trình viên kết hợp cả hai trong cùng một mã nguồn, giúp quản lý dễ dàng hơn, đặc biệt là trong các ứng dụng phức tạp.

Virtual DOM (Document Object Model ảo) là một bản sao nhẹ hơn của DOM thật. DOM thật là cấu trúc cây chứa tất cả các thành phần HTML trong trang web. Khi người dùng tương tác với ứng dụng (ví dụ: nhập văn bản, nhấn nút), ứng dụng sẽ thay đổi nội dung và DOM thật phải được cập nhật.

Tuy nhiên, việc cập nhật trực tiếp DOM thật có thể gây ra tình trạng chậm chạp và kém hiệu quả, đặc biệt trong các ứng dụng lớn với nhiều phần tử. Để giải quyết vấn

đề này, React sử dụng Virtual DOM, với mục đích chỉ những phần có sự thay đổi mới được cập nhật lên DOM thật, giúp tiết kiệm thời gian và tài nguyên [1].

2.2.4. Những lợi ích tuyệt vời mà ReactJS mang lại cho lập trình viên

Hiệu suất cao

ReactJS sử dụng Virtual DOM để tối ưu hóa hiệu suất của ứng dụng. Virtual DOM cho phép ReactJS cập nhật các thay đổi trên trang web một cách nhanh chóng và hiệu quả hơn so với cách truyền thống, giúp tăng tốc độ và hiệu suất của ứng dụng.

Tái sử dụng

ReactJS cho phép tái sử dụng các thành phần UI, giúp giảm thiểu thời gian và chi phí phát triển. Các thành phần UI có thể được sử dụng lại trong nhiều phần khác nhau của ứng dụng, giúp tăng tính linh hoạt và khả năng mở rộng của ứng dụng.

Dễ dàng quản lý trạng thái

ReactJS giúp quản lý trạng thái của ứng dụng một cách dễ dàng. Sử dụng State và Props, ReactJS cho phép các nhà phát triển quản lý trạng thái của các thành phần UI một cách chính xác và dễ dàng.

Hỗ trợ tốt cho SEO

ReactJS cho phép các nhà phát triển xây dựng ứng dụng web với khả năng tương thích tốt với SEO. Với sự hỗ trợ của các thư viện như React Helmet, ReactJS cho phép các nhà phát triển tùy chỉnh và quản lý các phần tử meta và title cho từng trang web.

Hỗ trợ đa nền tảng

ReactJS không chỉ được sử dụng để phát triển các ứng dụng web, mà còn được sử dụng để phát triển các ứng dụng di động với React Native. Sử dụng React Native, các nhà phát triển có thể xây dựng ứng dụng di động cho cả iOS và Android sử dụng cùng một mã nguồn [5].

2.2.5. Các tính năng nổi bật của ReactJS

Components

ReactJS cho phép phát triển ứng dụng web theo mô hình component. Các component là các phần tử UI độc lập có thể được tái sử dụng trong nhiều phần khác nhau của ứng dụng.

Virtual DOM

ReactJS sử dụng Virtual DOM để tối ưu hóa hiệu suất của ứng dụng. Virtual DOM là một bản sao của DOM được lưu trữ trong bộ nhớ và được cập nhật một cách nhanh chóng khi có thay đổi, giúp tăng tốc độ và hiệu suất của ứng dụng.

JSX

JSX là một ngôn ngữ lập trình phân biệt được sử dụng trong ReactJS để mô tả các thành phần UI. JSX kết hợp HTML và JavaScript, giúp cho việc viết mã dễ hiểu và dễ bảo trì hơn.

State và Props

ReactJS cho phép quản lý trạng thái của các thành phần UI thông qua State và Props. State là trạng thái của một thành phần được quản lý nội bộ bởi chính thành phần, trong khi Props là các giá trị được truyền vào từ bên ngoài để tùy chỉnh hoặc điều khiển hành vi của một thành phần.

Hỗ trợ tốt cho SEO

ReactJS hỗ trợ tốt cho việc tối ưu hóa SEO. Với các thư viện như React Helmet, các nhà phát triển có thể quản lý các phần tử meta và title cho từng trang web, giúp tăng khả năng tìm kiếm và tăng cường trải nghiệm người dùng.

Hỗ trợ đa nền tảng

ReactJS không chỉ được sử dụng để phát triển ứng dụng web, mà còn được sử dụng để phát triển ứng dụng di động với React Native. Sử dụng React Native, các nhà phát triển có thể xây dựng ứng dụng di động cho cả iOS và Android sử dụng cùng một mã nguồn.

Redux

Redux là một thư viện quản lý trạng thái cho các ứng dụng ReactJS. Redux giúp quản lý trạng thái của ứng dụng một cách chính xác và dễ dàng, đồng thời giúp tăng tính linh hoạt và khả năng mở rộng của ứng dụng [5].

2.2.6. Cách sử dụng ReactJS trong phát triển web

Bước 1 Cài đặt Node.js và npm

ReactJS được xây dựng trên nền tảng Node.js, do đó lập trình viên cần cài đặt Node.js và npm để phát triển ứng dụng ReactJS.

Bước 2 Tạo một ứng dụng React

Lập trình viên có thể tạo một ứng dụng React bằng cách sử dụng lệnh “create-react-app” trong Command Prompt hoặc Terminal.

Bước 3 Tạo các component

Tạo các component để xây dựng giao diện người dùng cho ứng dụng. Lập trình viên có thể tạo component bằng cách sử dụng class hoặc hàm.

Bước 4 Xây dựng giao diện người dùng

Sử dụng JSX để xây dựng giao diện người dùng cho ứng dụng. JSX là một ngôn ngữ phân biệt được sử dụng trong ReactJS để mô tả các thành phần UI.

Bước 5 Quản lý trạng thái

Sử dụng State và Props để quản lý trạng thái của các thành phần UI. State là trạng thái của một thành phần được quản lý nội bộ bởi chính thành phần, trong khi Props là các giá trị được truyền vào từ bên ngoài để tùy chỉnh hoặc điều khiển hành vi của một thành phần.

Bước 6 Kết nối với API

Sử dụng thư viện như Axios để kết nối với API và lấy dữ liệu từ server.

Bước 7 Build và triển khai ứng dụng

Sử dụng lệnh "npm run build" để build ứng dụng và triển khai trên môi trường sản phẩm [5].

2.3 Tìm hiểu về RESTful API

2.3.1. RESTful API



Hình 2.4 REST API

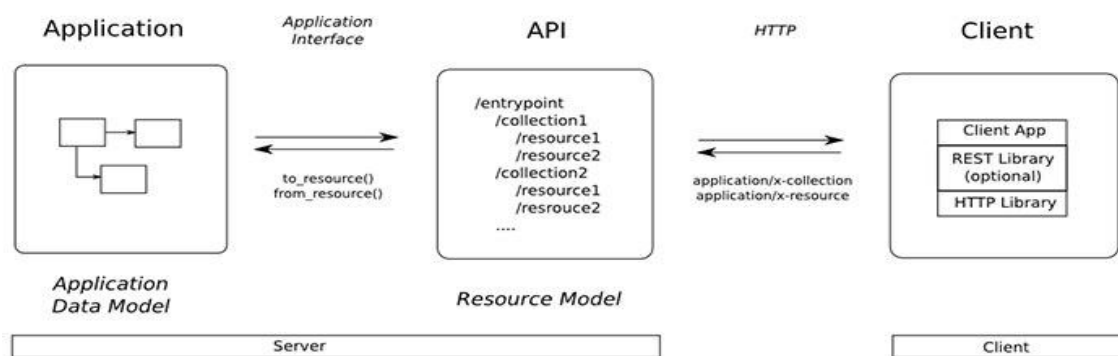
RESTful API là một tiêu chuẩn dùng trong việc thiết kế API cho các ứng dụng web (thiết kế Web services) để tiện cho việc quản lý các resource. RESTful API chú trọng vào tài nguyên hệ thống (tệp văn bản, ảnh, âm thanh, video, hoặc dữ liệu động...), bao gồm các trạng thái tài nguyên được định dạng và được truyền tải qua HTTP.

API (Application Programming Interface) là một tập các quy tắc và cơ chế mà theo đó, một ứng dụng hay một thành phần sẽ tương tác với một ứng dụng hay thành phần khác. API có thể trả về dữ liệu mà lập trình viên cần cho ứng dụng của mình ở những kiểu dữ liệu phổ biến như JSON hay XML.

REST (REpresentational State Transfer) là một dạng chuyển đổi cấu trúc dữ liệu, một kiểu kiến trúc để viết API. REST sử dụng phương thức HTTP đơn giản để tạo cho giao tiếp giữa các máy. Vì vậy, thay vì sử dụng một URL cho việc xử lý một số thông tin người dùng, REST gửi một yêu cầu HTTP như GET, POST, DELETE, đến một URL để xử lý dữ liệu.

Chức năng quan trọng nhất của REST là quy định cách sử dụng các HTTP method (như GET, POST, PUT, DELETE...) và cách định dạng các URL cho ứng dụng web để quản các resource. RESTful không quy định logic code ứng dụng và không giới hạn bởi ngôn ngữ lập trình ứng dụng, bất kỳ ngôn ngữ hoặc framework nào cũng có thể sử dụng để thiết kế một RESTful API [6].

2.3.2. RESTful hoạt động như thế nào



Hình 2.5 Cách hoạt động của RESTful API

REST hoạt động chủ yếu dựa vào giao thức HTTP. Các hoạt động cơ bản nêu trên sẽ sử dụng những phương thức HTTP riêng.

GET (SELECT): Trả về một Resource hoặc một danh sách Resource.

POST (CREATE): Tạo mới một Resource.

PUT (UPDATE): Cập nhật thông tin cho Resource.

DELETE (DELETE): Xóa một Resource.

Những phương thức hay hoạt động này thường được gọi là CRUD tương ứng với Create, Read, Update, Delete (Tạo, Đọc, Sửa, Xóa) [6].

2.3.3. API RESTful mang lại những lợi ích

Khả năng thay đổi quy mô

Các hệ thống triển khai API REST có thể thay đổi quy mô một cách hiệu quả vì REST tối ưu hóa các tương tác giữa client và máy chủ. Tình trạng phi trạng thái loại bỏ tải của máy chủ vì máy chủ không phải giữ lại thông tin yêu cầu của client trong quá khứ. Việc lưu bộ nhớ đệm được quản lý tốt sẽ loại bỏ một phần hoặc hoàn toàn một số tương tác giữa client và máy chủ. Tất cả các tính năng này hỗ trợ khả năng thay đổi quy mô mà không gây ra tắc nghẽn giao tiếp làm giảm hiệu suất.

Sự linh hoạt

Các dịch vụ web RESTful hỗ trợ phân tách hoàn toàn giữa client và máy chủ. Các dịch vụ này đơn giản hóa và tách riêng các thành phần máy chủ khác nhau để mỗi phần có thể phát triển độc lập. Các thay đổi ở nền tảng hoặc công nghệ tại ứng dụng

máy chủ không ảnh hưởng đến ứng dụng client. Khả năng phân lớp các chức năng ứng dụng làm tăng tính linh hoạt hơn nữa. Ví dụ: các nhà phát triển có thể thực hiện các thay đổi đối với lớp cơ sở dữ liệu mà không cần viết lại logic ứng dụng.

Sự độc lập

Các API REST không phụ thuộc vào công nghệ được sử dụng. Lập trình viên có thể viết cả ứng dụng client và máy chủ bằng nhiều ngôn ngữ lập trình khác nhau mà không ảnh hưởng đến thiết kế API. Và cũng có thể thay đổi công nghệ cơ sở ở hai phía mà không ảnh hưởng đến giao tiếp [2].

2.4 Tìm hiểu về NestJS

2.4.1. NestJS



Hình 2.6 NestJs

NestJS là một framework mã nguồn mở để phát triển ứng dụng server-side (backend applications) bằng ngôn ngữ TypeScript hoặc JavaScript. NestJS được xây dựng trên cơ sở của Node.js và sử dụng các khái niệm từ TypeScript để tạo ra một môi trường phát triển hiện đại và mạnh mẽ cho việc xây dựng các ứng dụng web và API.

Mục tiêu chính của NestJS là cung cấp một cấu trúc ứng dụng rõ ràng và dễ quản lý, giúp tăng tính bảo trì và sự tổ chức trong mã nguồn. Để đạt được điều này, NestJS triển khai mô hình kiến trúc lõi (core architecture) dựa trên các nguyên tắc của Angular, đặc biệt là sử dụng Dependency Injection (DI) và Modules (Các module) [3].

2.4.2. Cấu trúc của NestJS

Cấu trúc của NestJS được xây dựng dựa trên mô hình kiến trúc lõi (core architecture) giúp tạo ra một ứng dụng server-side (backend application) rõ ràng, dễ quản lý và dễ mở rộng. Cấu trúc NestJS thường được tổ chức thành các phần chính sau:

Module (Các module): Module là một phần cơ bản trong cấu trúc NestJS. Mỗi ứng dụng NestJS bao gồm ít nhất một module gốc (root module) và có thể có nhiều module con. Module là nơi tổ chức các thành phần của ứng dụng như Controllers, Providers và các thành phần khác. Mỗi module đại diện cho một phần chức năng cụ thể của ứng dụng.

Controller (Bộ điều khiển): Controllers là thành phần chịu trách nhiệm xử lý các yêu cầu HTTP từ phía client và trả về kết quả tương ứng. Controllers là nơi xử lý các request và trả về các response. Các phương thức của controller được chú thích

(decorated) bằng các decorator như `@Get()`, `@Post()`, `@Put()`, v.v., để chỉ định các route và phương thức HTTP tương ứng.

Provider (Các nhà cung cấp): Providers là thành phần chịu trách nhiệm cung cấp các dịch vụ cho ứng dụng. Đây có thể là các service, repository, logger, v.v. Providers sử dụng dependency injection để chèn vào các thành phần khác và có thể được sử dụng bởi các controllers hoặc các providers khác.

Middleware (Trung gian): Middleware là các hàm xử lý mà NestJS sử dụng để xử lý các yêu cầu HTTP trước khi chúng đến các route xử lý chính. Middleware có thể được sử dụng để thực hiện các thao tác chung như xác thực, ghi log, xử lý lỗi.

Filter (Bộ lọc): Filters được sử dụng để xử lý các exception (ngoại lệ) xảy ra trong ứng dụng. Filters cho phép lập trình viên xử lý và thay đổi response trước khi gửi về client khi có exception xảy ra.

Guard (Bảo vệ): Guards được sử dụng để kiểm tra xem một yêu cầu có thể được xử lý hoặc không. Guards cho phép lập trình viên thực hiện các kiểm tra xác thực hoặc kiểm tra quyền trước khi xử lý một yêu cầu.

Interceptor (Bộ chặn): Interceptors là các hàm xử lý mà NestJS sử dụng để chặn và thay đổi response trước khi response được gửi về client. Interceptors có thể được sử dụng để thực hiện các thao tác chung trên response trước khi đi ra ngoài.

Exception (Ngoại lệ): Exception handling (xử lý ngoại lệ) là một phần quan trọng của cấu trúc NestJS. Exception handling cho phép xử lý các exception xảy ra trong ứng dụng và trả về các thông báo lỗi thích hợp cho client [3].

2.4.3. Hướng dẫn cài đặt NestJS

Để cài đặt NestJS, lập trình viên cần đảm bảo đã cài đặt Node.js và npm (Node Package Manager) trước tiên. Sau đó, có thể sử dụng npm để cài đặt NestJS CLI (Command Line Interface) và tạo một dự án NestJS mới. Dưới đây là hướng dẫn cài đặt NestJS trên hệ điều hành Windows, macOS và Linux:

Bước 1: Cài đặt Node.js và npm

Trước tiên, hãy truy cập trang chủ của Node.js (<https://nodejs.org/>) để tải về phiên bản mới nhất và cài đặt Node.js và npm theo hướng dẫn trên trang web.

Bước 2: Cài đặt NestJS CLI

Sau khi cài đặt Node.js và npm thành công, mở cửa sổ dòng lệnh hoặc terminal trên máy tính và chạy lệnh sau để cài đặt NestJS CLI một cách toàn cầu (global):

```
npm install -g @nestjs/cli
```

Bước 3: Tạo một dự án NestJS mới

Sau khi cài đặt NestJS CLI, có thể sử dụng để tạo một dự án NestJS mới. Để tạo một dự án mới, hãy chạy lệnh sau trong cửa sổ dòng lệnh hoặc terminal:

```
nest new project-name
```

Trong đó, `project-name` là tên của dự án muốn tạo. NestJS CLI sẽ tạo ra một cấu trúc dự án NestJS cơ bản với các tập tin và thư mục cần thiết.

Bước 4: Chạy ứng dụng NestJS

Sau khi dự án NestJS đã được tạo thành công, có thể di chuyển vào thư mục dự án bằng cách chạy lệnh:

```
cd project-name
```

Sau đó, có thể chạy ứng dụng NestJS bằng lệnh sau:

```
npm run start
```

Hoặc nếu muốn tự động khởi động lại ứng dụng khi có thay đổi trong mã nguồn, có thể sử dụng lệnh:

```
npm run start:dev
```

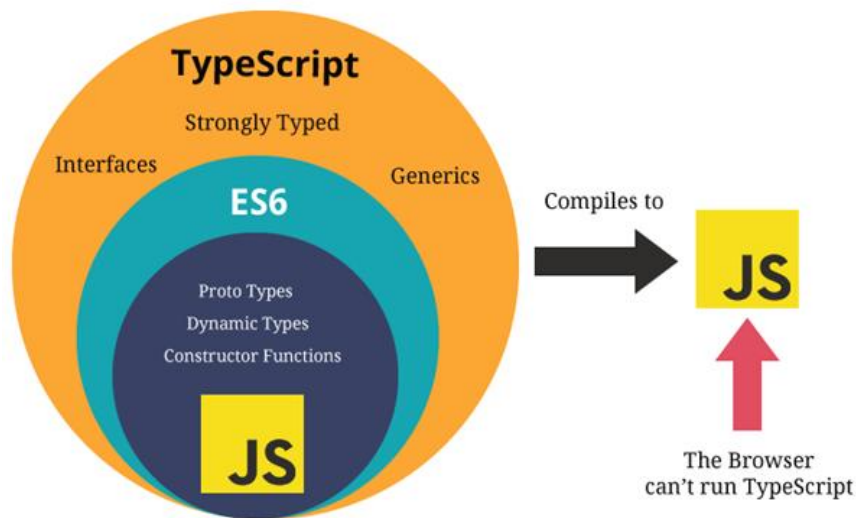
Sau khi ứng dụng đã chạy thành công người dùng có thể truy cập vào <http://localhost:3000> (hoặc cổng khác nếu đã thay đổi cài đặt) để xem ứng dụng NestJS [3].

2.5 Tìm hiểu về TypeScript

2.5.1. TypeScript

TypeScript (viết tắt là TS) là một ngôn ngữ lập trình mã nguồn mở (OOP) được phát triển và duy trì bởi Microsoft vào năm 2012. TypeScript được xem là một phần mở rộng của JavaScript, sử dụng cú pháp của JavaScript và bổ sung thêm các tính năng mạnh mẽ như kiểu tĩnh và hướng đối tượng để hỗ trợ Type (các kiểu dữ liệu) [7].

2.5.2. Ưu điểm của Typescript



Hình 2.7 TypeScript

TypeScript thật sự thuận tiện và hoàn toàn miễn phí

Đối với loại ngôn ngữ lập trình tĩnh như TypeScript, tất cả những số liệu, thông số của lập trình viên sẽ dễ dàng được lấy hơn nhờ IDE và trình biên dịch.

TypeScript hỗ trợ quá trình tìm kiếm giúp tiết kiệm thời gian kiểm tra lại code, không cần thông qua bất cứ một ai để có thể tìm thông tin dữ liệu, ngoài ra TypeScript làm giảm phần trăm va chạm lỗi trong thời gian vận hành.

Ngoài ra, đây cũng là một trong những phần mềm nổi trội được Microsoft hỗ trợ hoàn toàn miễn phí.

Thao tác nhanh chóng và đơn giản hơn

Ngôn ngữ TypeScript có thao tác khá đơn giản, tiết kiệm thời gian hơn nhưng lại đem đến kết quả tốt đến bất ngờ, khắc phục tình trạng xuất hiện lỗi và dễ đọc hơn.

Tái cấu trúc

Chắc chắn trong quá trình viết code, các lập trình viên sẽ thường xuyên mắc phải nhiều lỗi nhỏ và cần chỉnh sửa, việc sử dụng TypeScript sẽ giúp bước chỉnh sửa code trở nên dễ dàng hơn nhờ hiệu quả của lệnh Rename Symbol/Find All Occurrences [7].

Đối với các ngôn ngữ khác, khi muốn sửa chi tiết nào đó thì thường phải thay đổi luôn những tập tin khác nếu có liên quan hoặc sử dụng RegEx

Trong trường hợp người dùng TypeScript muốn nâng cấp hệ thống của mình (thêm hoặc xóa thuộc tính, đổi tên,...) thì TypeScript sẽ giúp tái cấu trúc lại sao cho phù hợp với nhu cầu tìm kiếm mà không gây náo loạn trong hệ thống. Trong trường hợp code không match được bất cứ dữ liệu nào thì sẽ được báo đến ngay để được xử lý ổn thỏa.

Giảm tỷ lệ mắc lỗi trong hệ thống

Nhờ vào việc cảnh báo lỗi ngay khi viết code, nên tỷ lệ mắc lỗi trong hệ thống khi sử dụng TypeScript khá thấp, TypeScript sẽ trả lại giá trị null hoặc gợi ý thay đổi chỉnh sửa. Mỗi lần chỉnh sửa sau khi được TypeScript báo lỗi thì phần trăm hệ thống hoạt động mà không mắc phải lỗi là rất cao, có thể dễ dàng thấy được TypeScript giúp người dùng tiết kiệm không ít thời gian để sửa lỗi.

Hợp nhất mã đơn giản

Sau khi hoàn thiện được một đoạn code và cho chúng chạy thử nghiệm, có thể ngay trong môi trường đó mọi thứ đều hoạt động trơn tru, nhưng có chắc được đoạn code đó cũng sẽ hoạt động tốt khi ở trong môi trường điều kiện khác.

Một trong những điểm mạnh của TypeScript là có thể hợp nhất mã một cách đơn giản để có thể dễ dàng kiểm tra đánh giá đoạn mã vừa mới cho ra đời kia bằng cách sử dụng Typedef – kiểm tra biên dịch.

Lại một lần nữa, TypeScript lại giúp người dùng tiết kiệm thời gian và công sức.

Hỗ trợ tối ưu hóa quy trình làm việc

TypeScript sẽ không khuyến khích người dùng nhảy bước, thực hiện sai thao tác. TypeScript khuyến khích người dùng đưa ra quyết định về kiểu dữ liệu khi sử dụng ngôn ngữ kiểu tĩnh trước khi thực hiện thao tác, các bước tiếp theo. Chính vì những quy luật như thế sẽ giúp cho các lập trình viên tối ưu được quy trình làm việc hiệu quả hơn [7].

2.5.3. Nhược điểm của Typescript

Bất kỳ ngôn ngữ nào cũng có điểm yếu và hạn chế và TypeScript cũng vậy điểm hạn chế của TypeScript là:

Bắt buộc dùng biên dịch

Để có thể vận hành một tệp TypeScript với đuôi .js trên nền tảng Node.js bắt buộc phải dùng trình biên dịch để có thể sử dụng.

Bước thiết lập công kênh

Trước khi có thể sử dụng được TypeScript, cần đảm bảo rằng máy chủ Node.js, trình thử nghiệm và webpack đều có thể hoạt động với TypeScript, nếu không thì sẽ không sử dụng được.

Bên cạnh đó, mỗi khi apply thêm bất kỳ library nào như Redux, React và Styled-Component thì cũng phải thêm Typedef vào.

Chỉ là phần ngôn ngữ mở rộng hỗ trợ

Sau cùng, chức năng của TypeScript cũng chỉ là để biên dịch về JavaScript, TypeScript không phải là một ngôn ngữ có thể vận hành độc lập và cũng đồng thời không thể thay thế được vai trò của JavaScript. Chức năng của TypeScript bị giới hạn bởi chức năng của JavaScript, TypeScript chỉ là được nâng cấp từ điểm yếu của JavaScript.

Chỉ với mỗi TypeScript, người dùng không thể nào hoàn thiện được các công đoạn của dự án, công dụng của TypeScript chỉ thực sự nổi bật khi được kết hợp nhuần nhuyễn và tối ưu với các ngôn ngữ, nền tảng và tool khác [7].

2.6 Tìm hiểu về MongoDB

2.6.1. MongoDB



Hình 2.8 MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu mã nguồn mở, là CSDL thuộc NoSql và được hàng triệu người sử dụng.

MongoDB là một database hướng tài liệu (document), các dữ liệu được lưu trữ trong document kiểu JSON thay vì dạng bảng như CSDL quan hệ nên truy vấn sẽ rất nhanh.

Với CSDL quan hệ chúng ta có khái niệm bảng, các cơ sở dữ liệu quan hệ (như MySQL hay SQL Server...) sử dụng các bảng để lưu dữ liệu thì với MongoDB chúng ta sẽ dùng khái niệm là collection thay vì bảng.

Các collection trong MongoDB được cấu trúc rất linh hoạt, cho phép các dữ liệu lưu trữ không cần tuân theo một cấu trúc nhất định.

Thông tin liên quan được lưu trữ cùng nhau để truy cập truy vấn nhanh thông qua ngôn ngữ truy vấn MongoDB [8].

2.6.2. Ưu điểm của MongoDB

Do MongoDB sử dụng lưu trữ dữ liệu dưới dạng Document JSON nên mỗi một collection sẽ có các kích cỡ và các document khác nhau, linh hoạt trong việc lưu trữ dữ liệu, nên người dùng muốn gì thì cứ insert vào thoải mái.

Dữ liệu trong MongoDB không có sự ràng buộc lẫn nhau, không có join như trong RDBMS nên khi insert, xóa hay update MongoDB không cần phải mất thời gian kiểm tra xem có thỏa mãn các ràng buộc dữ liệu như trong RDBMS.

MongoDB rất dễ mở rộng (Horizontal Scalability). Trong MongoDB có một khái niệm cluster là cụm các node chứa dữ liệu giao tiếp với nhau, khi muốn mở rộng hệ thống ta chỉ cần thêm một node vào cluster:

Trường dữ liệu “_id” luôn được tự động đánh index (chỉ mục) để tốc độ truy vấn thông tin đạt hiệu suất cao nhất.

Khi có một truy vấn dữ liệu, bản ghi được cached lên bộ nhớ Ram, để phục vụ lượt truy vấn sau diễn ra nhanh hơn mà không cần phải đọc từ ổ cứng.

Hiệu năng cao: Tốc độ truy vấn (find, update, insert, delete) của MongoDB nhanh hơn hẳn so với các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Với một lượng dữ liệu đủ lớn thì thử nghiệm cho thấy tốc độ insert của MongoDB có thể nhanh tới gấp 100 lần so với MySQL [8].

2.6.3. Nhược điểm của MongoDB

MongoDB không có các tính chất ràng buộc như trong RDBMS nên khi thao tác với mongoDB thì phải hết sức cẩn thận.

Tồn bộ nhớ do dữ liệu lưu dưới dạng key-value, các collection chỉ khác về value do đó key sẽ bị lặp lại. Không hỗ trợ join nên dễ bị dư thừa dữ liệu.

Khi insert/update/remove bản ghi, MongoDB sẽ chưa cập nhật ngay xuống ổ cứng, mà sau 60 giây MongoDB mới thực hiện ghi toàn bộ dữ liệu thay đổi từ RAM xuống ổ cứng điều này sẽ là nhược điểm vì sẽ có nguy cơ bị mất dữ liệu khi xảy ra các tình huống như mất điện... [8].

CHƯƠNG: 3 HIỆN THỰC HÓA NGHIÊN CỨU

3.1 Mô tả bài toán

Trong bối cảnh hiện đại hóa nông nghiệp và sự phát triển của công nghệ số, việc xây dựng một sàn thương mại điện tử chuyên về nông sản cho tỉnh Trà Vinh là rất cần thiết. Hiện nay, nhiều nông dân và hợp tác xã tại Trà Vinh gặp khó khăn trong việc tiêu thụ nông sản do thiếu kênh bán hàng trực tuyến, thiếu kết nối với người tiêu dùng và doanh nghiệp ở các địa phương khác. Đồng thời, người tiêu dùng cũng gặp khó khăn trong việc tìm kiếm nguồn nông sản sạch, rõ nguồn gốc với giá cả hợp lý.

Do đó, việc xây dựng một website thương mại điện tử chuyên cung cấp và tiêu thụ nông sản của tỉnh Trà Vinh sẽ mang lại giải pháp tối ưu, giúp kết nối giữa nhà sản xuất và người tiêu dùng, đồng thời đẩy mạnh chuyển đổi số trong lĩnh vực nông nghiệp địa phương.

Hệ thống cho phép người dùng (khách hàng) đăng ký tài khoản, quản lý thông tin cá nhân và thực hiện các chức năng chính như:

 Tìm kiếm, xem thông tin chi tiết và đặt mua các sản phẩm nông sản (rau củ, trái cây)

 Theo dõi trạng thái đơn hàng và giao hàng.

 Đánh giá và bình luận sản phẩm sau khi mua hàng.

 Đối với người bán : hệ thống cho phép đăng bán sản phẩm, cập nhật số lượng và giá cả, quản lý đơn hàng và thống kê.

 Đối với quản trị viên: có thể quản lý toàn bộ danh sách sản phẩm nông sản, người dùng (bao gồm khách hàng và nhà cung cấp), đơn hàng phát sinh trên hệ thống, thống kê.

Sàn thương mại điện tử nông sản Trà Vinh không chỉ góp phần thúc đẩy tiêu thụ nông sản địa phương mà còn tạo điều kiện để nông dân tiếp cận với thương mại số, nâng cao giá trị nông sản và đáp ứng nhu cầu ngày càng cao của thị trường.

3.2 Yêu cầu chức năng

Khách hàng

- Đăng ký/đăng nhập
- Tìm kiếm sản phẩm theo tên, danh mục, giá.
- Lọc sản phẩm theo các tiêu chí: giá, loại nông sản.
- Xem trang chi tiết sản phẩm: hình ảnh, mô tả.

- Thêm sản phẩm vào giỏ hàng, chỉnh sửa số lượng và thanh toán.
- Xem giỏ hàng, tính tổng tiền, phí vận chuyển (nếu có).
- Xem lịch sử đơn hàng với trạng thái: đang xử lý, đang giao, đã giao, đã hủy.
- Xem chi tiết từng đơn: sản phẩm, số lượng, tổng tiền, thời gian.
- Đánh giá sao (1-5) cho từng sản phẩm đã mua.

Nhà cung cấp

- Thêm mới sản phẩm: tên, ảnh, mô tả, thành phần, giá bán, số lượng tồn kho.
- Sửa, xoá sản phẩm đã đăng.
- Cập nhật trạng thái sản phẩm: còn hàng, hết hàng, tạm ngừng bán..
- Xem danh sách đơn hàng mới, đang xử lý, đã giao.
- Cập nhật trạng thái đơn hàng (Chờ xác nhận, Đã xác nhận, Đang giao hàng,

Giao thất bại, Hoàn thành, Đã hủy).

- Thống kê doanh thu.

Quản trị

- Xem danh sách tất cả khách hàng và nhà cung cấp.
- Khóa/mở khóa tài khoản người dùng.
- Quản lý danh mục nông sản (thêm,sửa).
- Xem và quản lý tất cả sản phẩm trên sàn.
- Xem chi tiết đơn hàng, trạng thái xử lý của đơn hàng.
- Duyệt sản phẩm mà nhà cung cấp đăng bán.
- Xem tất cả các đánh giá về sản phẩm, có thể xóa nếu không phù hợp.
- Thống kê doanh thu.

3.3 Thiết kế cơ sở dữ liệu

Collection user

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string",
  "password": "string",
  "phone": "string",
  "address": "string",
  "avatarUrl": "string",
  "role": "customer | supplier | admin",
}
```

Collection storeprofiles

```
{
  "_id": "ObjectId",
  "userId": "ObjectId (ref: users)",
  "storeName": "string",
  "phone": "string",
  "address": "string",
  "imageUrl": "string",
  "isApproved": false,
}
```

Collection products

```
{
  "_id": "ObjectId",
  "name": "string",
  "slug": "string (unique)",
  "description": "string",
  "price": number,
  "quantity": number,
  "unitType": "string",
  "unitDisplay": "string",
}
```



```
"stock": number,  
"origin": "string",  
"images": ["string"],  
"categoryId": "ObjectId (ref: categories)",  
"supplierId": "ObjectId (ref: users)",  
"isActive": true,  
"status": "pending | approved | rejected | out_of_stock",  
"createdAt": "Date",  
"updatedAt": "Date"  
}
```

Collection categories

```
{  
  "_id": "ObjectId",  
  "name": "string",  
  "description": "string",  
  "image": "string",  
}
```

Collection orders

```
{  
  "_id": "ObjectId",  
  "customerId": "ObjectId (ref: users)",  
  "items": [  
    {  
      "productId": "ObjectId (ref: products)",  
      "supplierId": "ObjectId (ref: users)",  
      "quantity": number,  
      "price": number,  
      "isReviewed": false  
    }  
  ],  
  "totalAmount": number,  
  "shippingAddress": "string",  
}
```

```
    "paymentMethod": "string",
    "status": "Chưa thanh toán | Đã thanh toán",
    "shippingStatus": "Chờ xác nhận | Đã xác nhận | Đang giao hàng | Giao thất
bại | Hoàn thành",
    "isPaid": false,
    "isReviewed": false,
    "createdAt": "Date",
    "updatedAt": "Date"
}
```

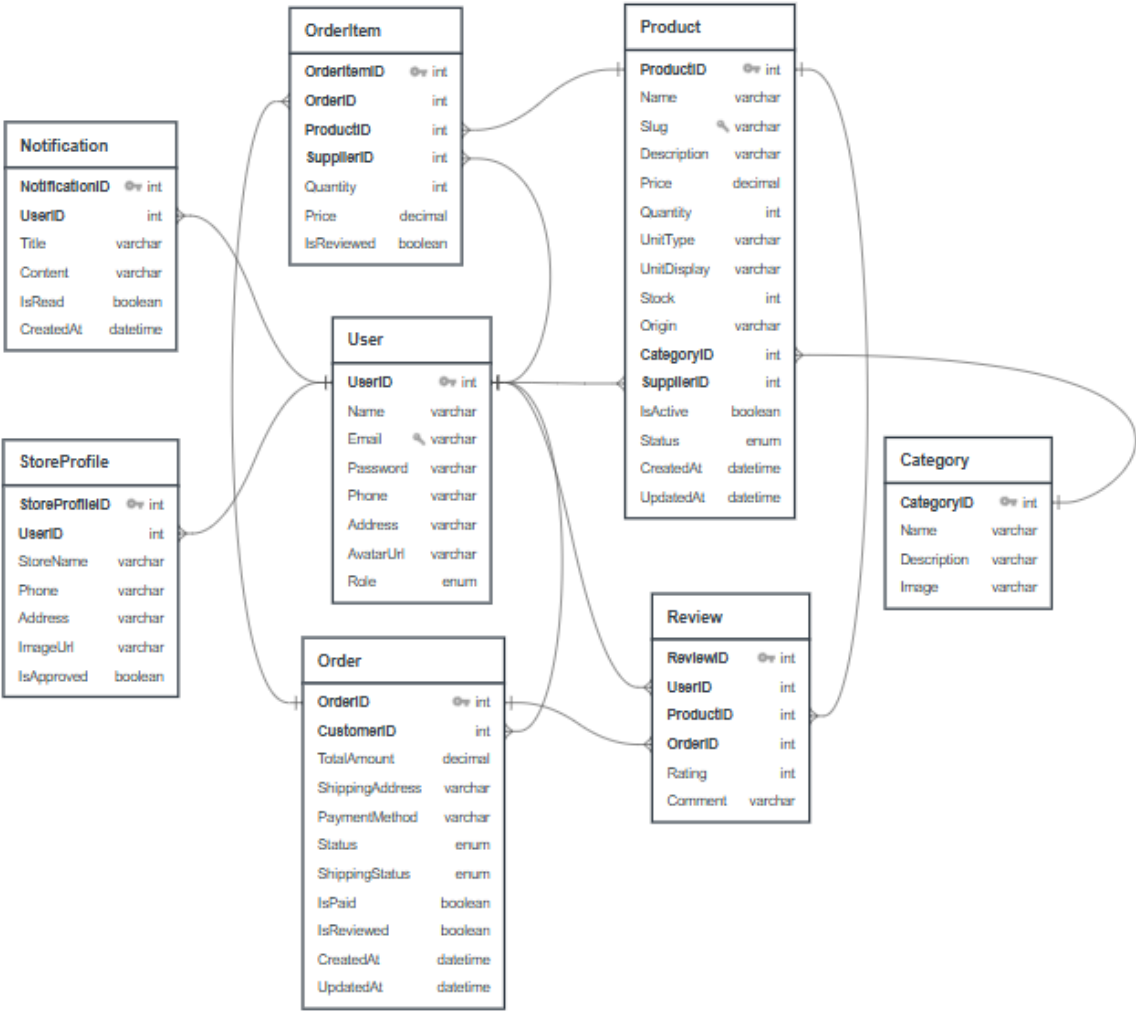
Collection reviews

```
{
  "_id": "ObjectId",
  "userId": "ObjectId (ref: users)",
  "productId": "ObjectId (ref: products)",
  "orderId": "ObjectId (ref: orders)",
  "rating": number,
  "comment": "string",
}
```

Collection notifications

```
{
  "_id": "ObjectId",
  "userId": "ObjectId (ref: users)",
  "title": "string",
  "content": "string",
  "isRead": "boolean (default: false)",
  "createdAt": "Date (default: Date.now)"
}
```

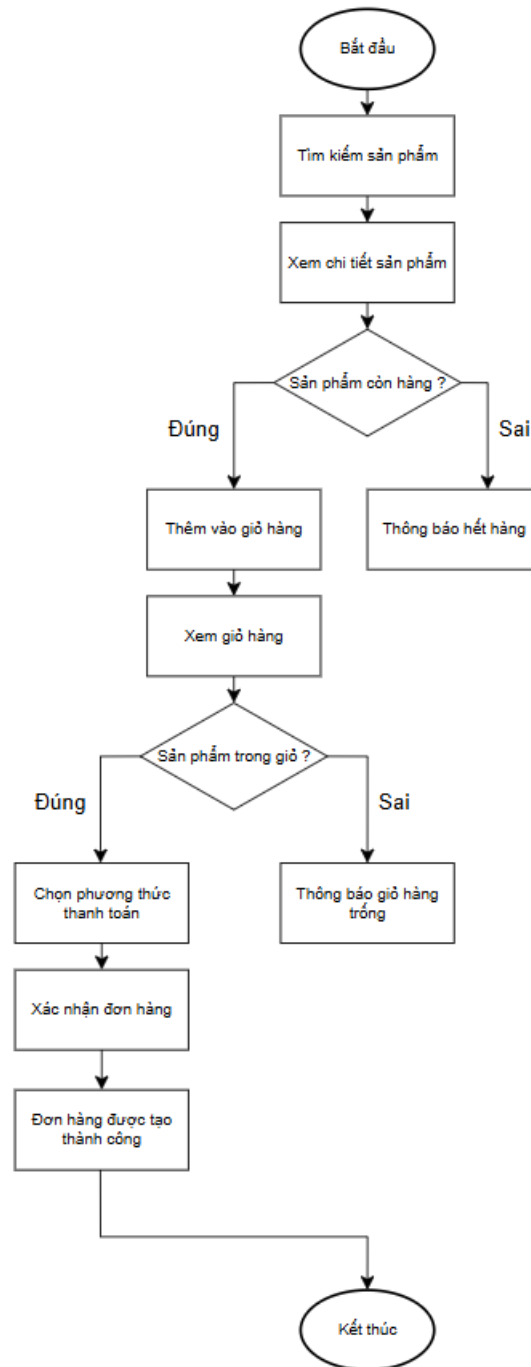
3.4 Mô hình thực thể



Hình 3.1 Mô hình thực thể

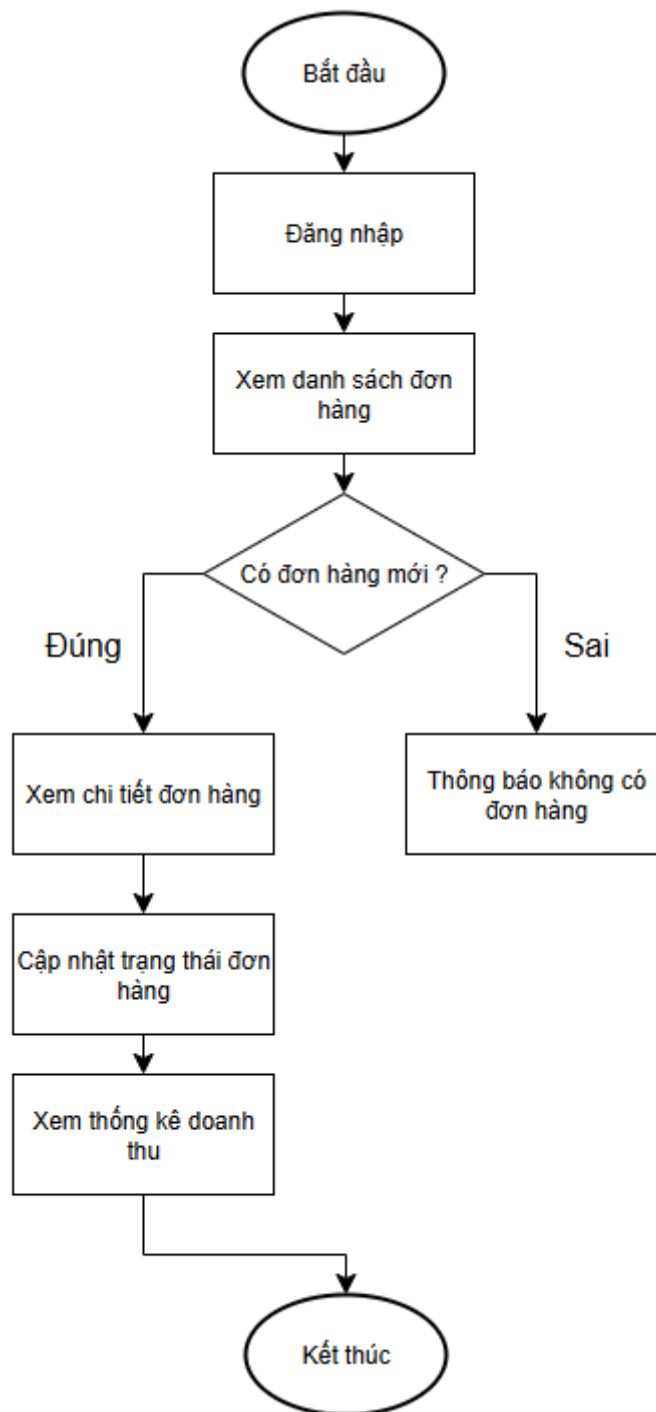
3.5 Biểu đồ flowchart

3.5.1. Biểu đồ luồng khách hàng đặt hàng



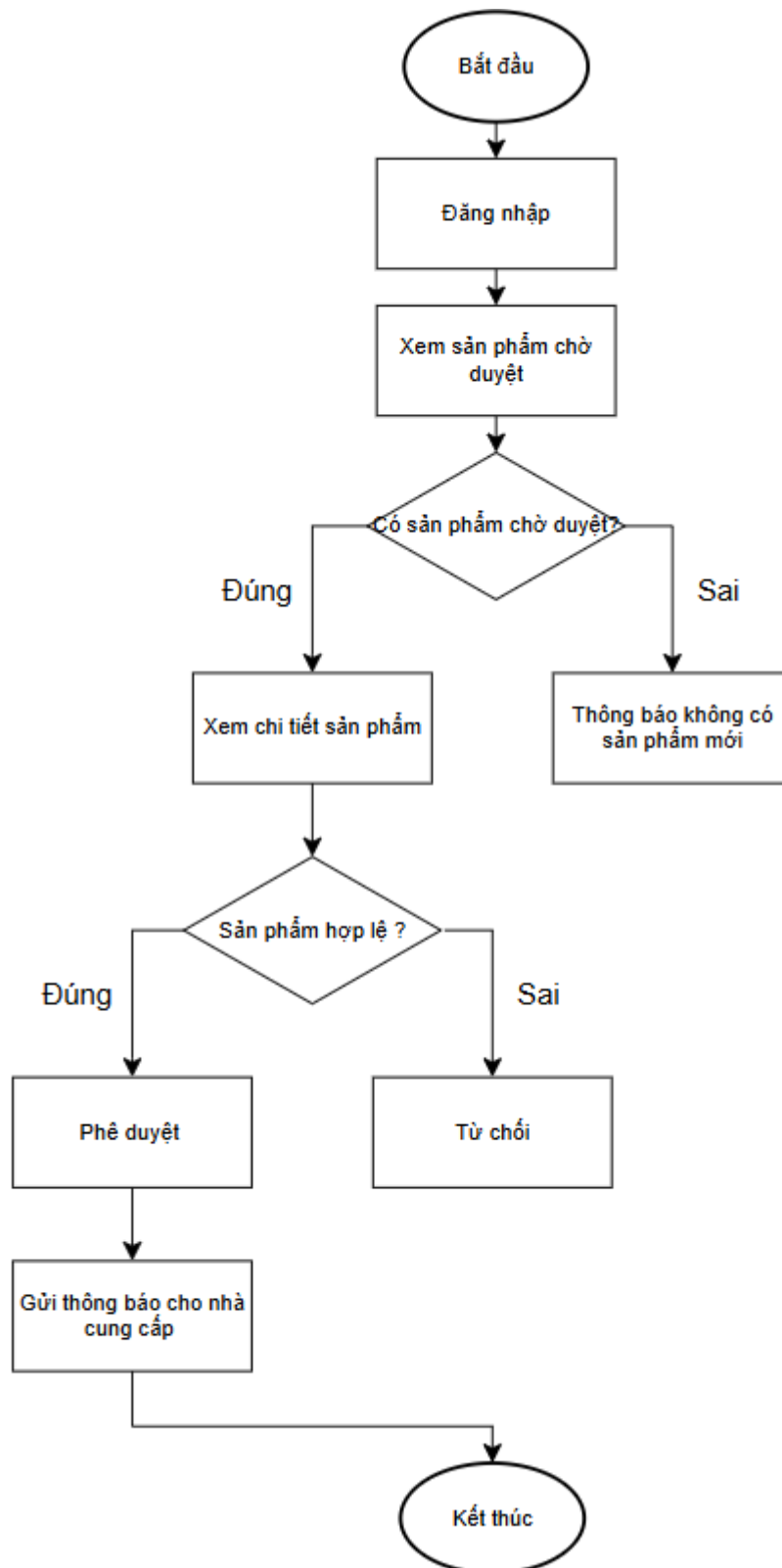
Hình 3.2 Biểu đồ luồng xử lý đặt hàng

3.5.2. Biểu đồ luồng nhà cung cấp xử lý đơn hàng



Hình 3.3 Biểu đồ luồng xử lý đơn hàng

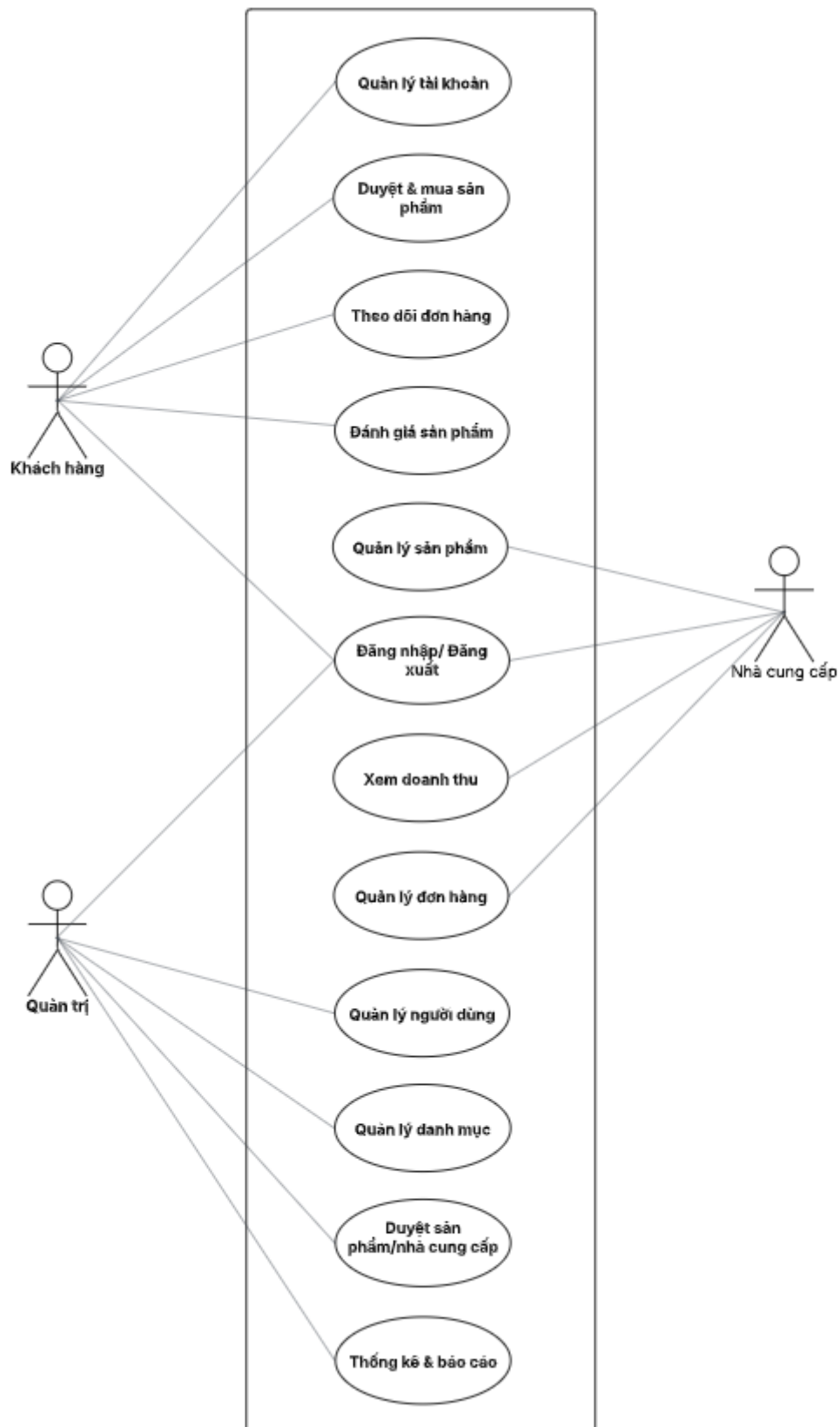
3.5.3. Biểu đồ luồng quản trị viên duyệt sản phẩm



Hình 3.4 Biểu đồ luồng xử lý duyệt sản phẩm

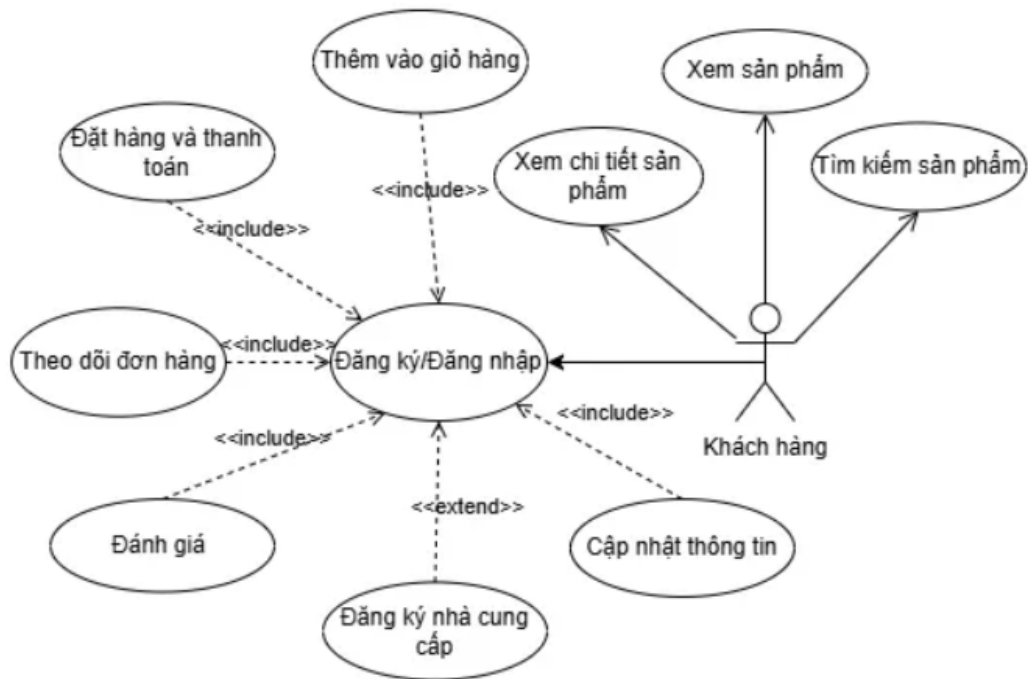
3.6 Biểu đồ Use Case

3.6.1. Biểu đồ Use Case tổng quát



Hình 3.5 Biểu đồ use case tổng quát

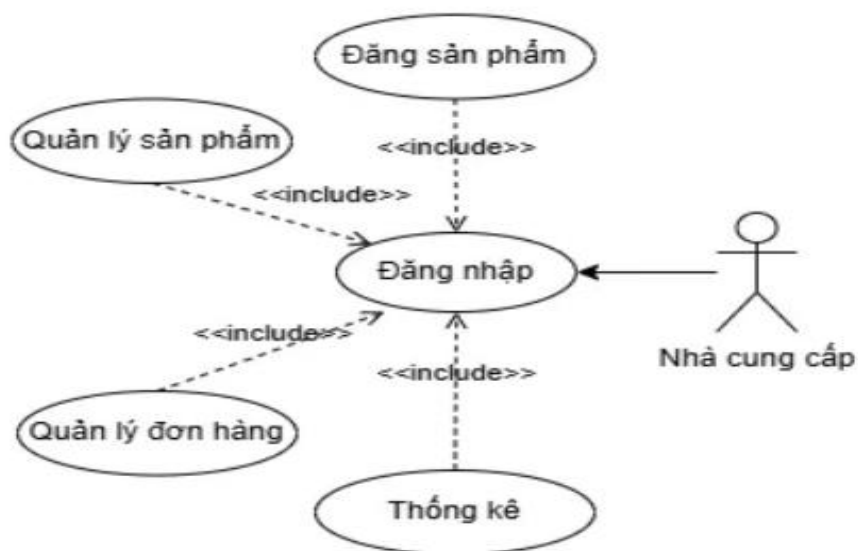
3.6.2. Biểu đồ Use Case tác nhân người dùng



Hình 3.6 Biểu đồ use case người dùng

Khi người dùng truy cập vào hệ thống và chưa có tài khoản thì có thể thực hiện các thao tác như : Đăng ký tài khoản, cập nhật thông tin cá nhân, đăng ký thành nhà cung cấp, tìm kiếm sản phẩm, xem sản phẩm, xem chi tiết sản phẩm, thêm vào giỏ hàng, đặt hàng và thanh toán, theo dõi đơn hàng và đánh giá sản phẩm.

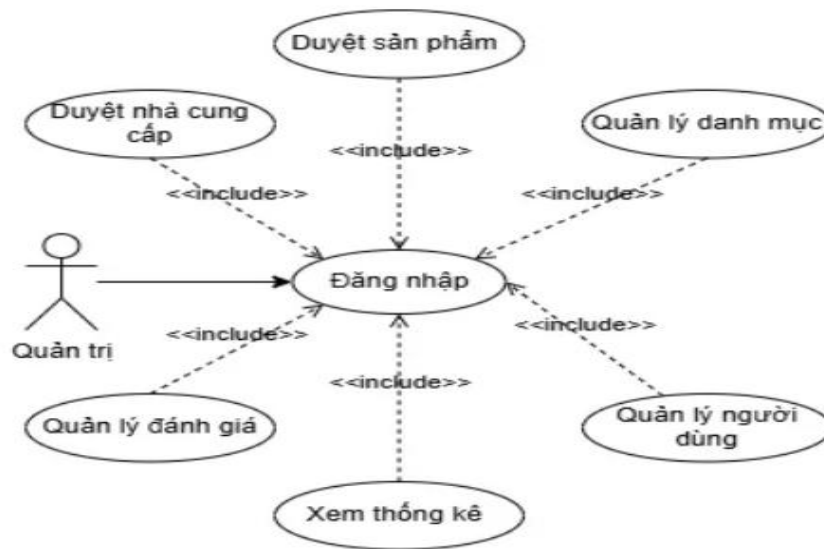
3.6.3. Biểu đồ Use Case tác nhân nhà cung cấp



Hình 3.7 Biểu đồ use case nhà cung cấp

Khi truy cập hệ thống với quyền nhà cung cấp thì có thể thực hiện các thao tác như: Đăng tải sản phẩm, quản lý sản phẩm (thêm, sửa, xóa), quản lý đơn hàng của mình và thống kê.

3.6.4. Biểu đồ Use Case tác nhân quản trị



Hình 3.8 Biểu đồ use quản trị

Quản trị : là người sử dụng hệ thống với quyền quản trị cao nhất

Đăng nhập: quản trị cần đăng nhập để truy cập và thực hiện các chức năng quản lý.

Quản lý thông tin người dùng: chức năng này cho phép quản trị có thể xem, xóa, khóa tài khoản người dùng.

Duyệt hồ sơ nhà cung cấp: Kiểm duyệt hồ sơ người dùng gửi để trở thành nhà cung cấp.

Duyệt sản phẩm: Đảm bảo sản phẩm hợp lệ trước khi hiển thị trên trang bán hàng

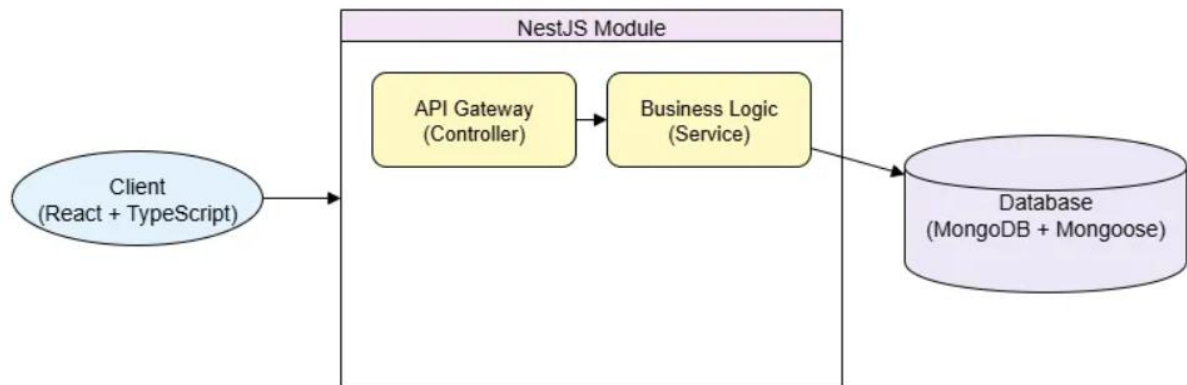
Quản lý danh mục sản phẩm: thêm mới và cập nhật danh mục.

Quản lý đơn hàng: xem tất cả các đơn hàng, xem chi tiết từng đơn (sản phẩm, người mua, trạng thái thanh toán, vận chuyển)

Quản lý đánh giá: quản trị có thể xem các đánh giá của tất cả sản phẩm và có thể xóa nếu nội dung đánh giá không phù hợp.

Quản lý thống kê: quản trị viên sẽ xem thống kê về đơn đặt hàng, hủy đơn.

3.7 Kiến trúc hệ thống



Hình 3.9 Kiến trúc hệ thống

Client (Giao diện người dùng - Frontend)

Công nghệ sử dụng: Được xây dựng bằng thư viện React kết hợp với TypeScript, cho phép phát triển giao diện một cách linh hoạt, dễ bảo trì và dễ mở rộng.

Chức năng: Là nơi người dùng tương tác trực tiếp với hệ thống. Giao diện bao gồm các chức năng như:

Đăng nhập, đăng ký

Tìm kiếm và xem thông tin sản phẩm.

Thêm sản phẩm vào giỏ hàng, đặt hàng và thanh toán.

Theo dõi tình trạng đơn hàng.

Giao tiếp với backend: Thông qua các RESTful API sử dụng giao thức HTTP (GET, POST, PUT, DELETE). Các request sẽ được gửi từ trình duyệt đến hệ thống backend và backend sẽ trả về dữ liệu phù hợp để client hiển thị.

Backend (Máy chủ - NestJS)

Công nghệ sử dụng: NestJS – một framework Node.js mạnh mẽ và có cấu trúc rõ ràng, hỗ trợ lập trình hướng đối tượng và mô-đun hóa.

Cấu trúc mô-đun hóa:

Hệ thống được tổ chức thành nhiều module độc lập như UserModule, ProductModule, OrderModule, mỗi module đảm nhận một chức năng cụ thể, giúp hệ thống dễ mở rộng và bảo trì.

Thành phần chính:

API Gateway (Controller):

Nhận request từ phía frontend

Xác thực dữ liệu đầu vào (nếu cần)

Định tuyến và gọi đến các phương thức nghiệp vụ bên trong service.

Business Logic (Service):

Chứa toàn bộ logic xử lý nghiệp vụ như: kiểm tra số lượng tồn kho, xử lý đơn hàng, tính tổng tiền, xác thực người dùng, xử lý thanh toán.

Thực hiện thao tác với cơ sở dữ liệu thông qua các model do Mongoose cung cấp.

Cơ sở dữ liệu (MongoDB + Mongoose)

MongoDB:

Là hệ quản trị cơ sở dữ liệu NoSQL, lưu trữ dữ liệu dưới dạng document (bản ghi dạng JSON).

Thích hợp cho các ứng dụng có cấu trúc dữ liệu linh hoạt và yêu cầu hiệu suất cao.

Mongoose:

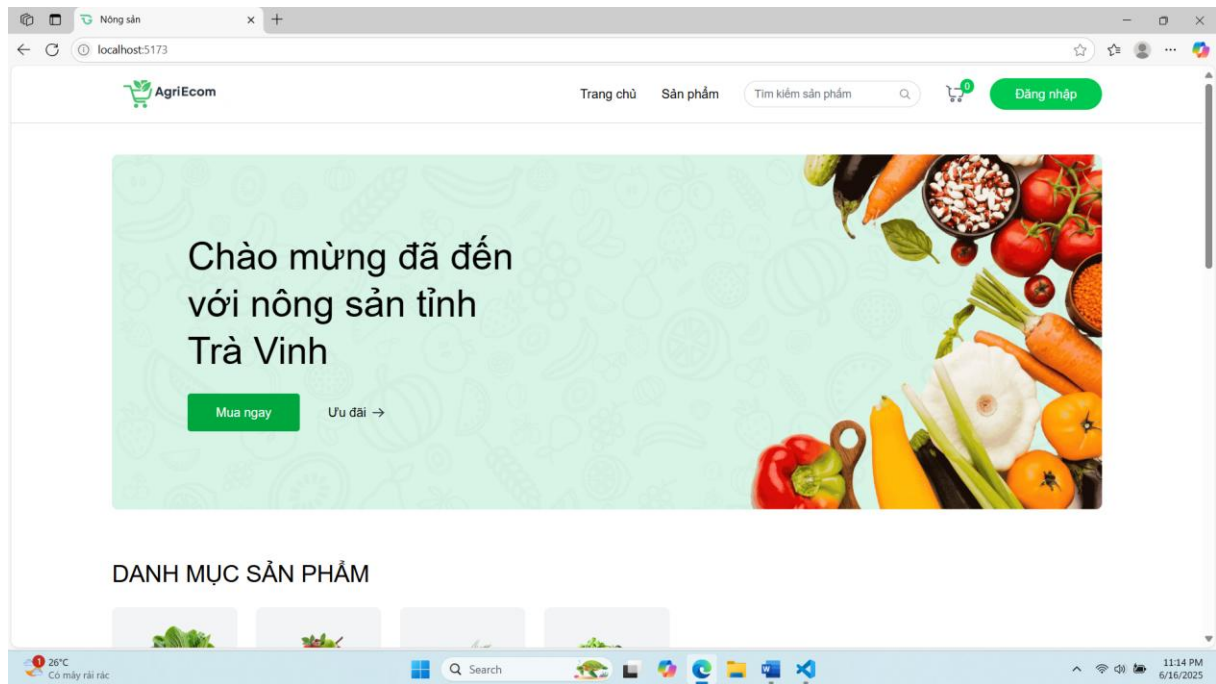
Là thư viện giúp giao tiếp giữa ứng dụng Node.js (NestJS) và MongoDB một cách dễ dàng.

Cung cấp khả năng định nghĩa schema (cấu trúc dữ liệu) thực hiện các thao tác CRUD và đảm bảo tính nhất quán của dữ liệu.

CHƯƠNG: 4 KẾT QUẢ NGHIÊN CỨU

4.1 Giao diện người dùng

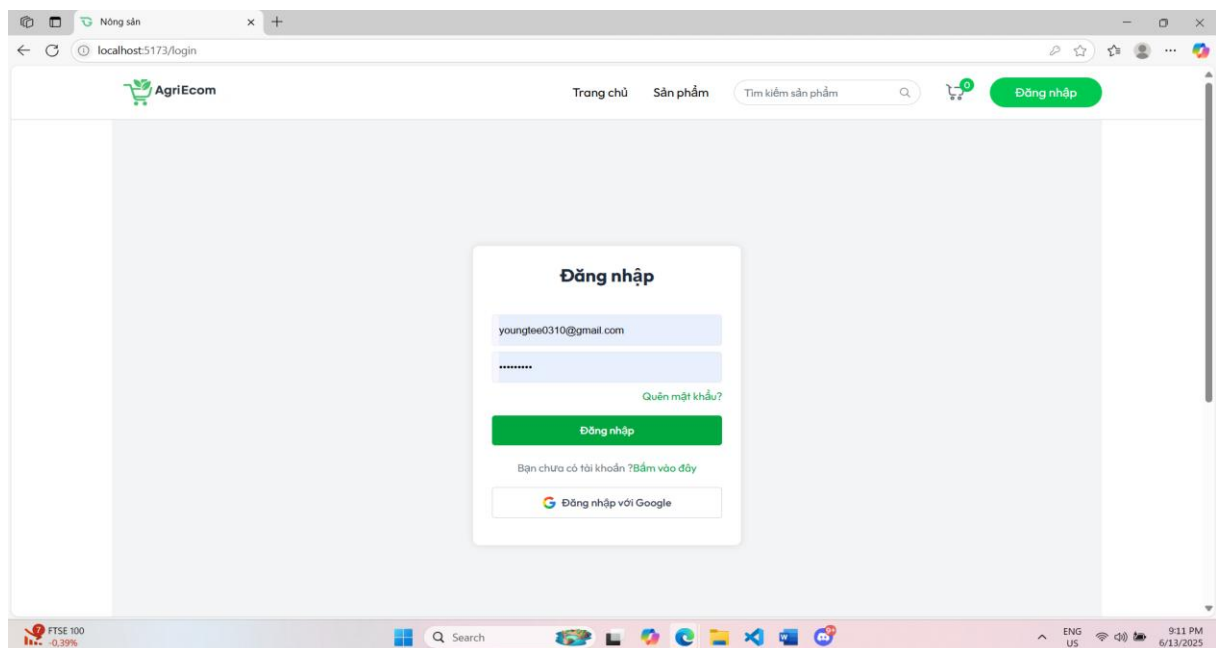
4.1.1. Giao diện trang chủ



Hình 4.1 Giao diện trang chủ

Mô tả: Giao diện trang chủ bao gồm banner chính nổi bật với thông điệp chào mừng, bên dưới hiển thị danh mục sản phẩm và sản phẩm bán chạy.

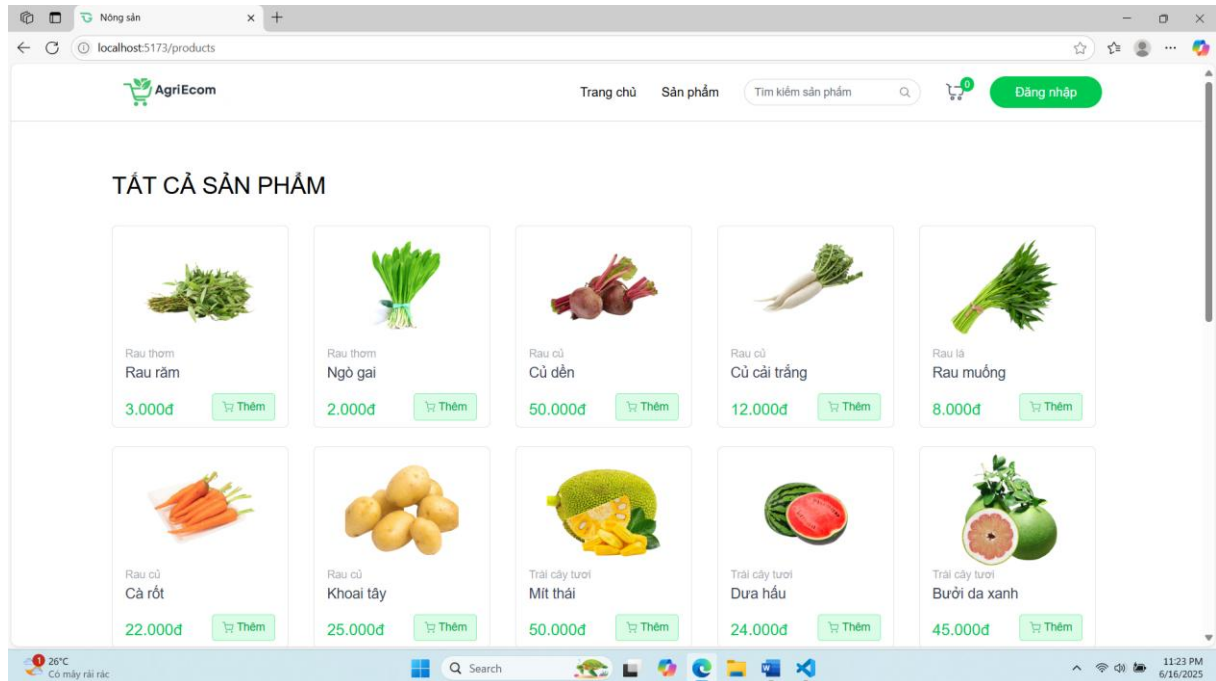
4.1.2. Giao diện đăng nhập



Hình 4.2 Giao diện đăng nhập

Mô tả: Giao diện đăng nhập cho phép người dùng đăng nhập để sử dụng hệ thống nếu chưa có thì người dùng có thể đăng ký hoặc đăng nhập bằng tài khoản Google và cũng có thể đặt lại mật khẩu nếu người dùng đã quên.

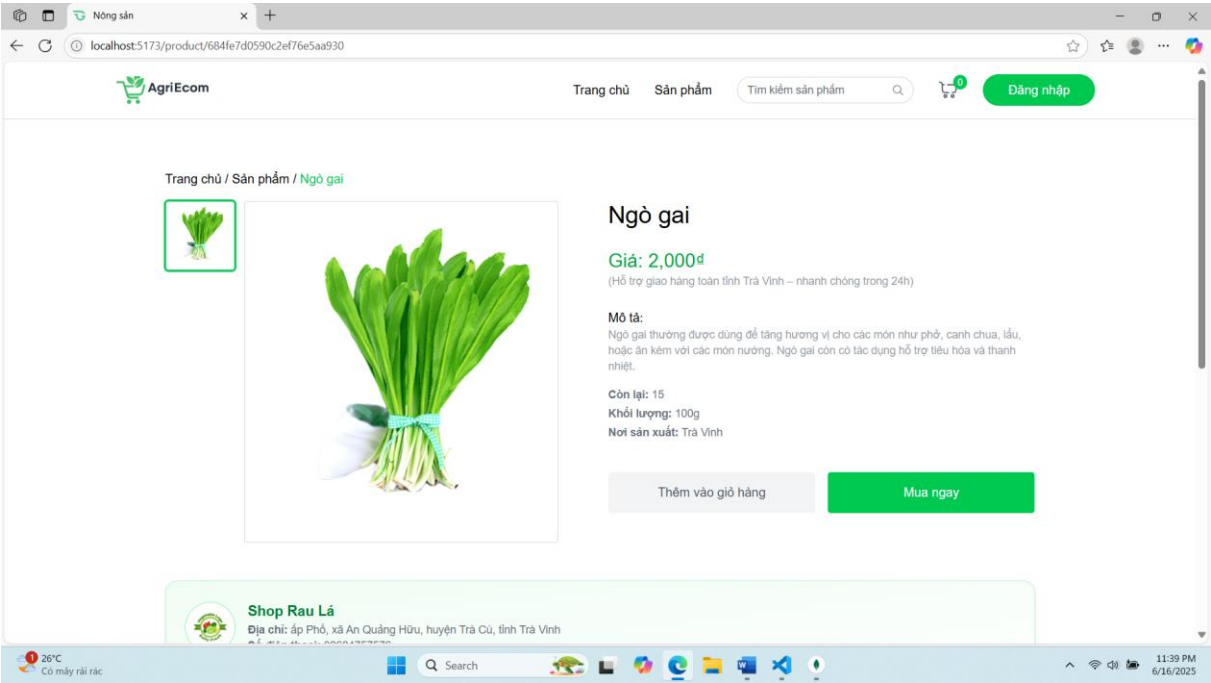
4.1.3. Giao diện sản phẩm



Hình 4.3 *Giao diện sản phẩm*

Mô tả : Giao diện hiển thị tất cả sản phẩm bao gồm tên, danh mục, giá sản phẩm và có thể thêm sản phẩm vào giỏ hàng giúp người dùng nhanh chóng lưu sản phẩm yêu thích để tiến hành thanh toán sau.

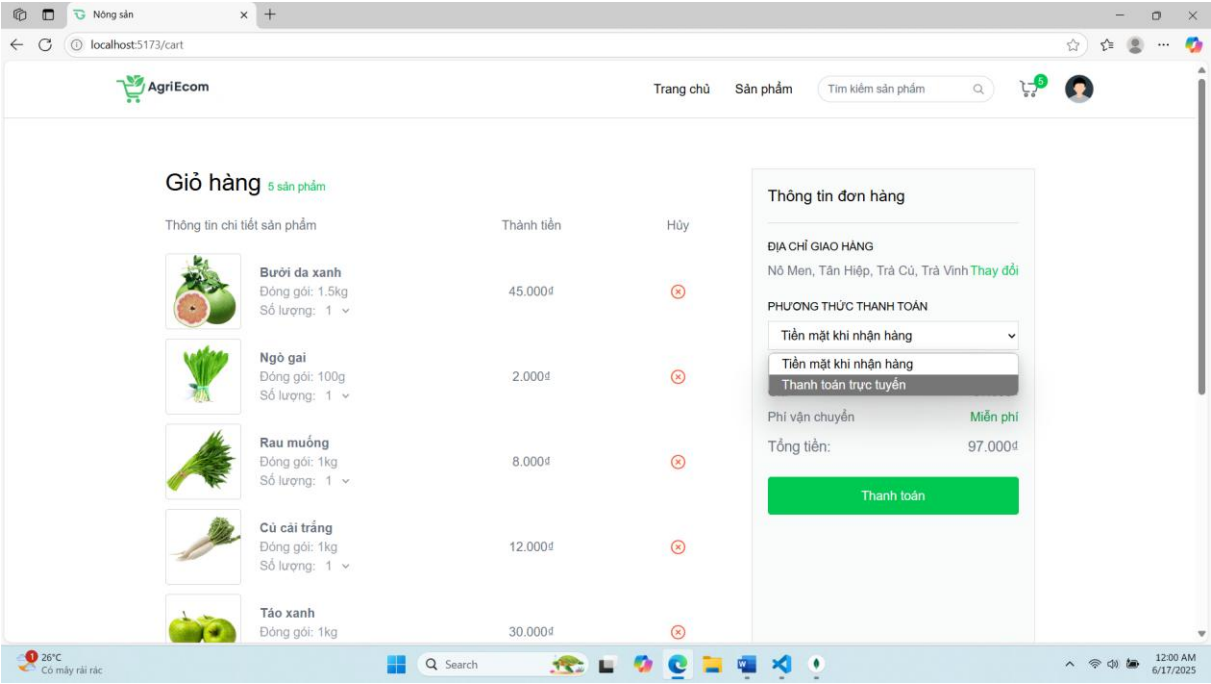
4.1.4. Giao diện chi tiết sản phẩm



Hình 4.4 Giao diện chi tiết sản phẩm

Mô tả: Giao diện chi tiết sản phẩm hiển thị một số thông tin về sản phẩm như: tên, giá, mô tả, khối lượng, nơi sản xuất và có thể thêm sản phẩm vào giỏ hàng hoặc mua ngay, phía dưới là thông tin nhà cung cấp và có phần đánh giá sản phẩm.

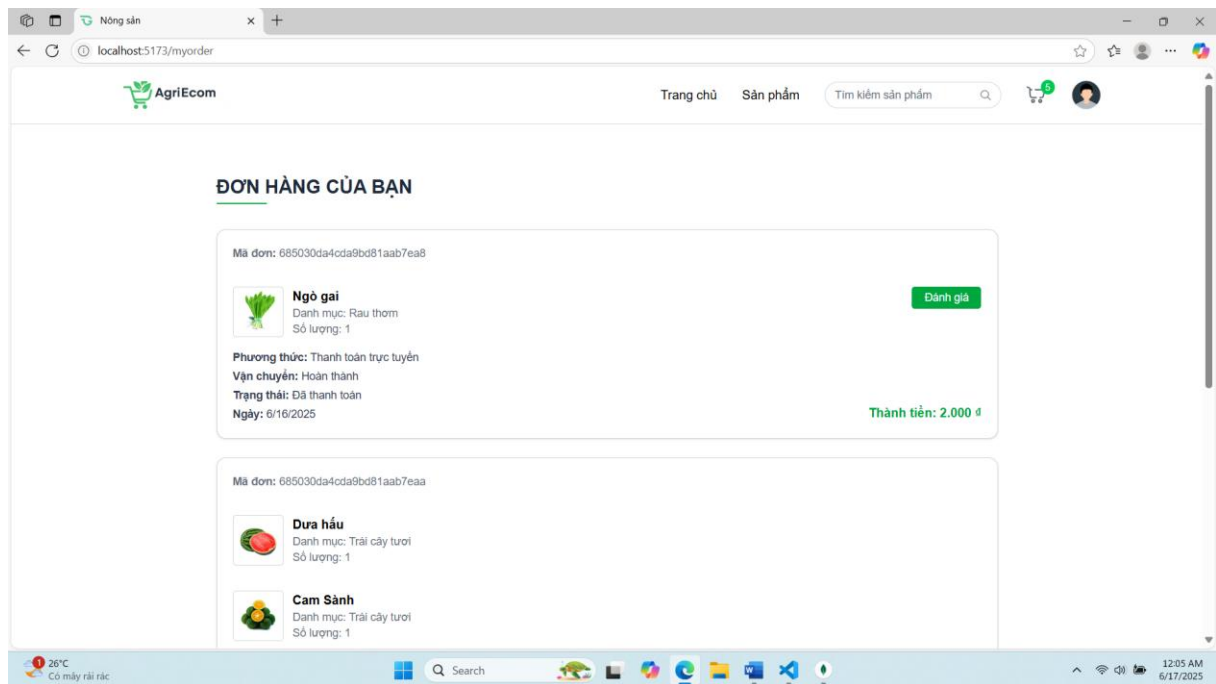
4.1.5. Giao diện giỏ hàng



Hình 4.5 Giao diện giỏ hàng

Mô tả: Giao diện giỏ hàng bao gồm các thông tin về sản phẩm, số lượng, tổng tiền người dùng có thể lựa chọn 2 hình thức thanh toán (thanh toán khi nhận hàng và thanh toán trực tuyến) khi chọn thanh toán khi nhận hàng thì sẽ chuyển qua trang đơn hàng còn thanh toán trực tuyến thì sẽ hiển thị mã QR và phải thanh toán trong vòng 2 phút không thì đơn hàng sẽ bị hủy.

4.1.6. Giao diện đơn hàng



Hình 4.6 *Giao diện đơn hàng*

Mô tả: Khi người đặt hàng thành công thì có thể xem thông tin về sản phẩm và theo dõi quá trình giao hàng, ngày đặt và khi hoàn thành xong đơn hàng thì có thể đánh giá sản phẩm.

4.2 Giao diện nhà cung cấp

4.2.1. Giao diện đăng ký nhà cung cấp

The screenshot shows a web browser window with the URL `localhost:5173/myorder`. The page displays the 'Đăng ký nhà cung cấp' (Supplier Registration) form. The form includes fields for 'Tên cửa hàng' (Store Name), 'Số điện thoại' (Phone Number), and 'Địa chỉ' (Address). There is a section for 'Ảnh đại diện' (Profile Picture) with a 'Choose File' button and a note 'No file chosen'. A green 'GỬI ĐĂNG KÝ' (Submit Registration) button is at the bottom. The background shows a list of products: 'Ngô gai' (Green Corn), 'Dưa hấu' (Watermelon), and 'Cam Sành' (Cam Sành). The total amount is 'Thành tiền: 2.000 đ'.

Hình 4.7 *Giao diện đăng ký nhà cung cấp*

Mô tả: Giao diện này người dùng có thể điền thông tin để đăng ký trở thành nhà cung cấp để đăng bán sản phẩm.

4.2.2. Giao diện thêm sản phẩm

The screenshot shows a web browser window with the URL `localhost:5173/seller/add-product`. The page displays the 'THÊM SẢN PHẨM MỚI' (Add New Product) form. The form includes fields for 'Tên sản phẩm' (Product Name) with the value 'Cam', 'Danh mục' (Category) with the value 'Trái cây tươi', and 'Mô tả sản phẩm' (Product Description) with the text 'Trái cam là một loại quả có mùi, vỏ ngoài màu cam và thịt mọng nước, ngọt dịu hoặc chua nhẹ, giàu vitamin C và khoáng chất, thường được dùng làm nước ép, ăn tươi hoặc chế biến món ăn.' There are also fields for 'Giá (VNĐ)' (Price) with the value '25000', 'Tồn kho' (Inventory) with the value '10', 'Số lượng' (Quantity) with the value '01', and 'Đơn vị (kg, gói...)' (Unit) with the value 'kg'. The 'Xuất xứ' (Origin) field has the value 'Huyện Công Long, Trà Vinh'. There is a section for 'Hình ảnh' (Image) with a 'Choose Files' button and a note 'orange_image.png'. The background shows a list of products: 'Thêm sản phẩm', 'Danh sách sản phẩm', 'Sản phẩm bị từ chối', 'Đơn hàng', and 'Danh thu'.

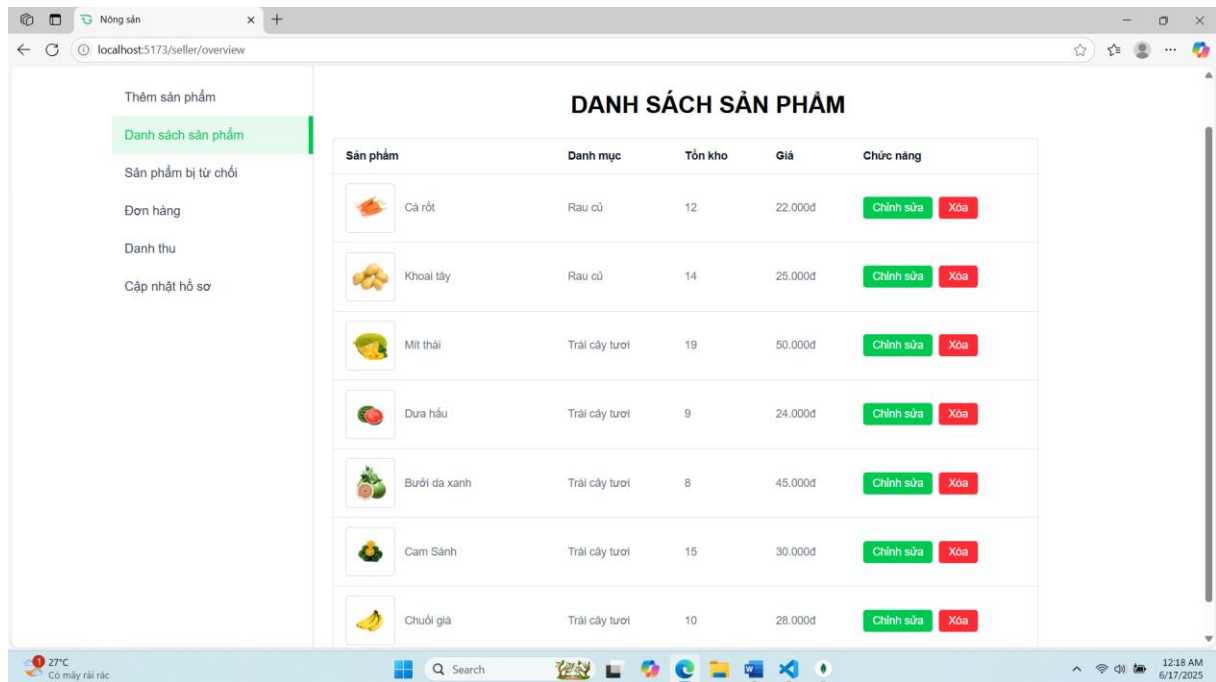
Hình 4.8 *Giao diện thêm sản phẩm*

Mô tả: Giao diện thêm mới sản phẩm cho phép nhà cung cấp thêm mới sản phẩm.

Xây dựng sàn thương mại điện tử lĩnh vực nông sản ở Trà Vinh

phẩm với các thông tin như: tên, danh mục, mô tả, giá, số lượng, hình ảnh khi bấm gửi thì đợi quản trị phê duyệt sản phẩm nếu từ chối thì sẽ nhận được thông báo.

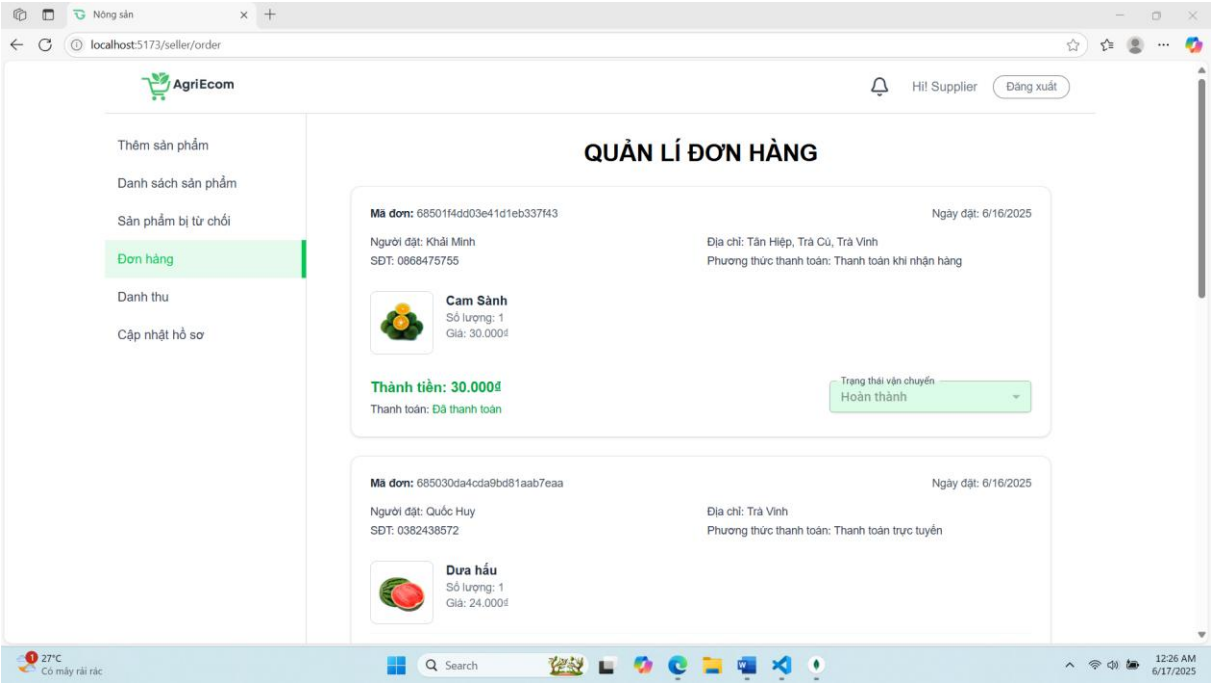
4.2.3. Giao diện quản lý sản phẩm



Hình 4.9 *Giao diện quản lý sản phẩm*

Mô tả: Giao diện này hiển thị tất cả sản phẩm của nhà cung cấp đã được duyệt và có thể chỉnh sửa và xóa sản phẩm.

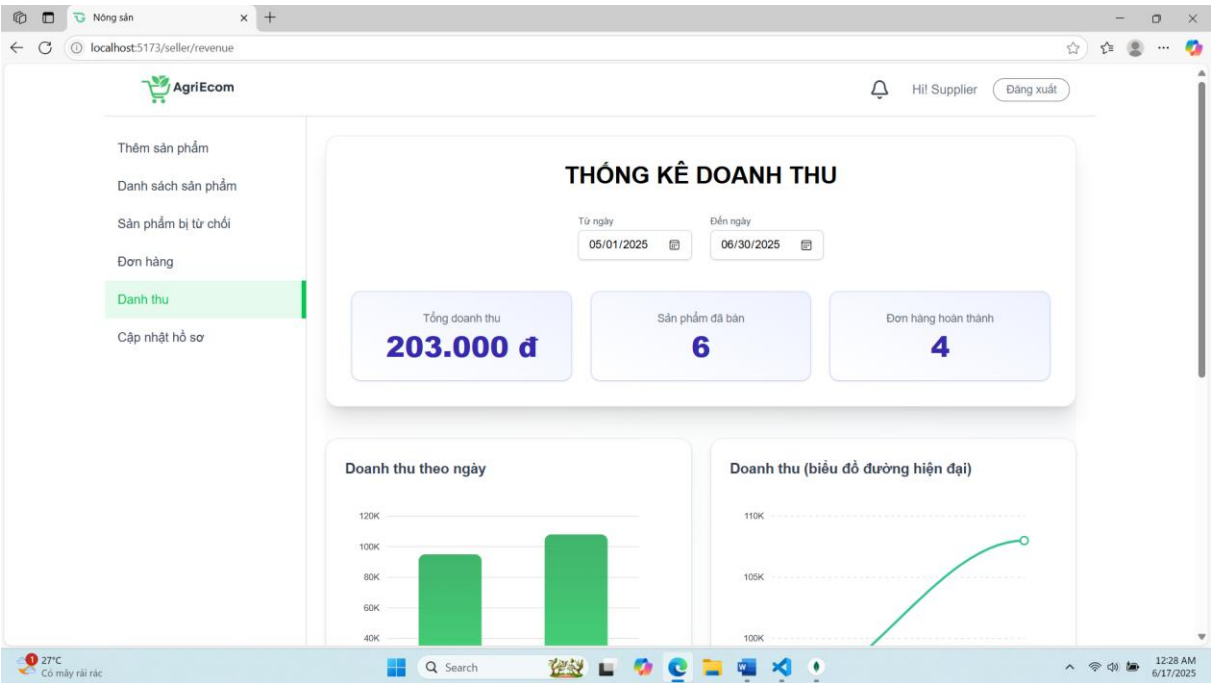
4.2.4. Giao diện quản lý đơn hàng



Hình 4.10 Giao diện quản lý đơn hàng

Mô tả: Giao diện quản lý đơn hàng cho phép nhà cung cấp xem thông tin người đặt sản phẩm và có thể cập nhật trạng thái giao hàng của sản phẩm.

4.2.5. Giao diện thống kê doanh thu

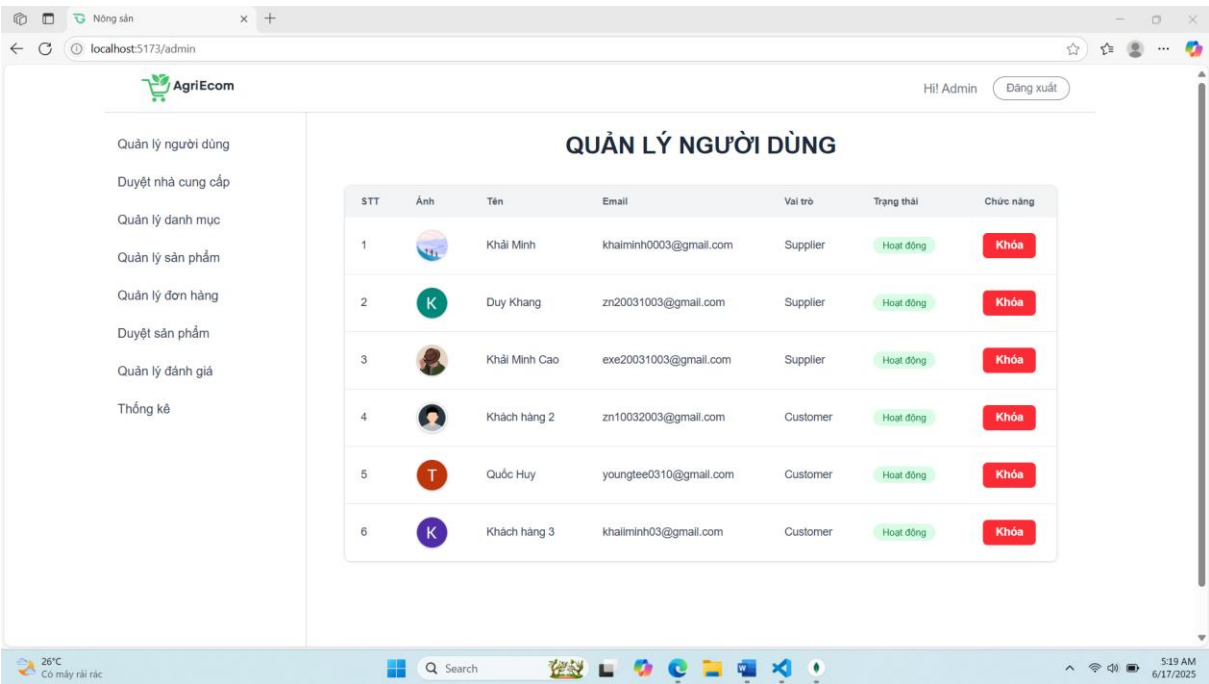


Hình 4.11 Giao diện thống kê doanh thu

Mô tả: Thống kê doanh thu theo ngày và các sản phẩm bán chạy, trạng thái đơn hàng, tổng doanh thu, tổng sản phẩm đã bán, sản phẩm đã giao thành công.

4.3 Giao diện quản trị

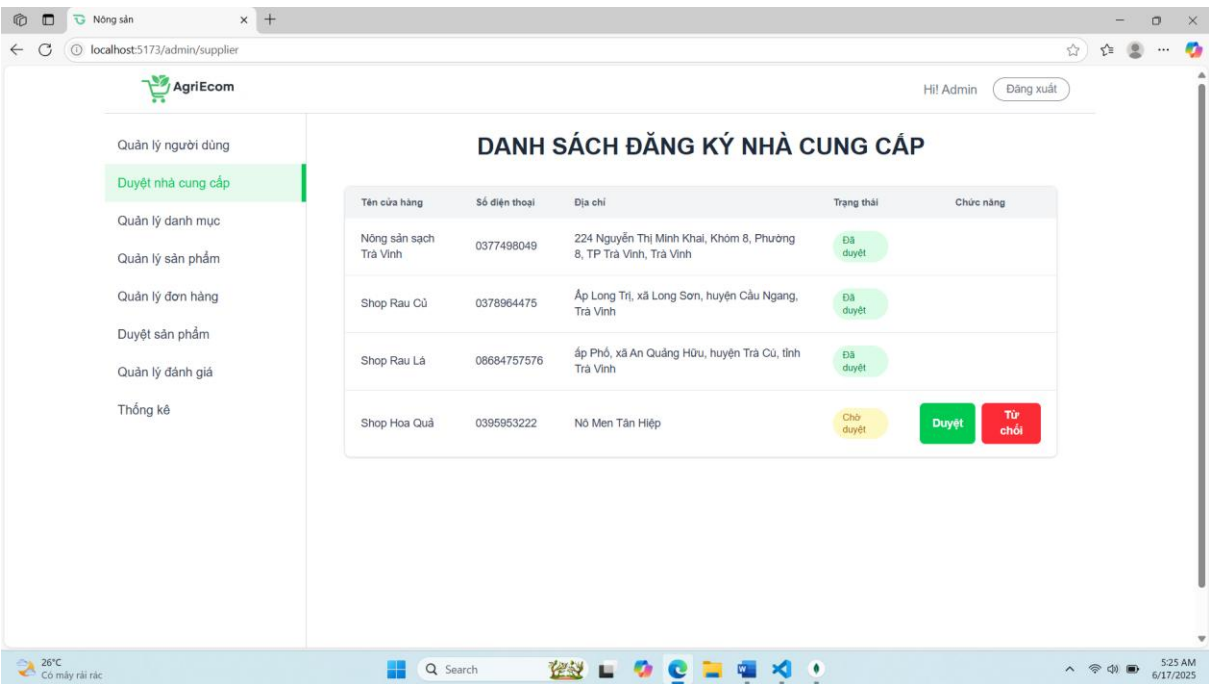
4.3.1. Giao diện quản lý người dùng



Hình 4.12 Giao diện quản lý người dùng

Mô tả: Quản trị viên có thể xem thông tin người dùng có thể khóa tài khoản người dùng nếu vi phạm.

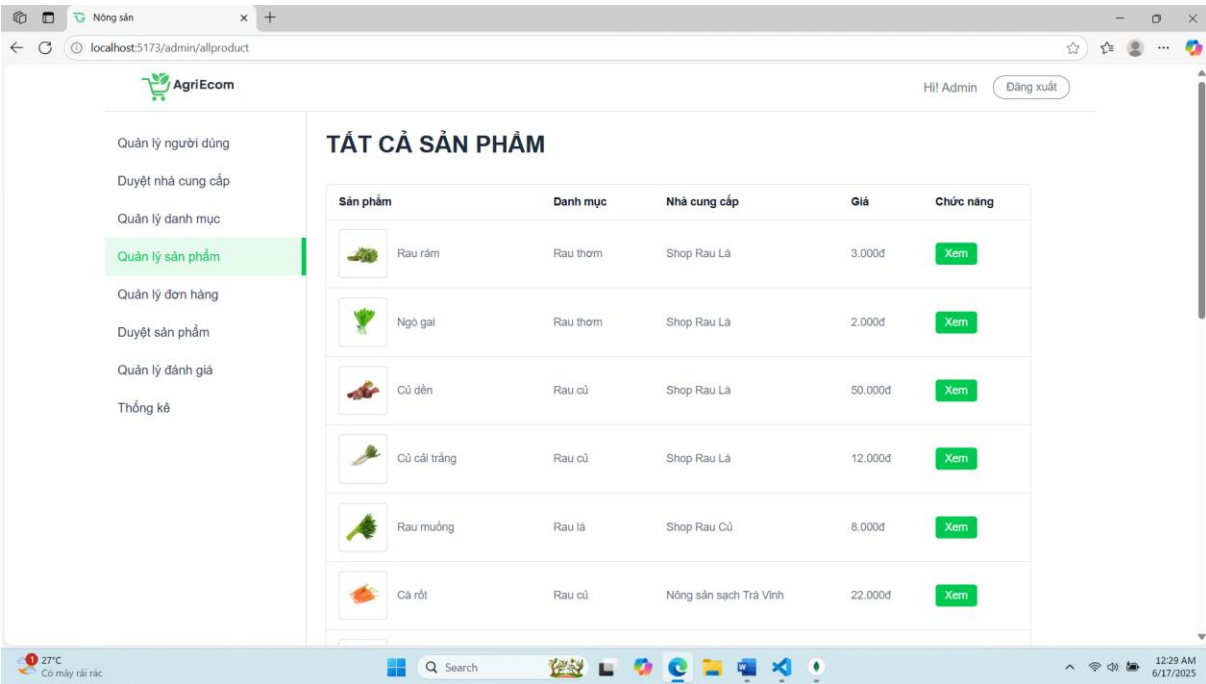
4.3.2. Giao diện duyệt nhà cung cấp



Hình 4.13 Giao diện duyệt nhà cung cấp

Mô tả: Giao diện này cho phép quản trị có thể duyệt người dùng thành nhà cung cấp để có thể đăng tải sản phẩm lên sàn.

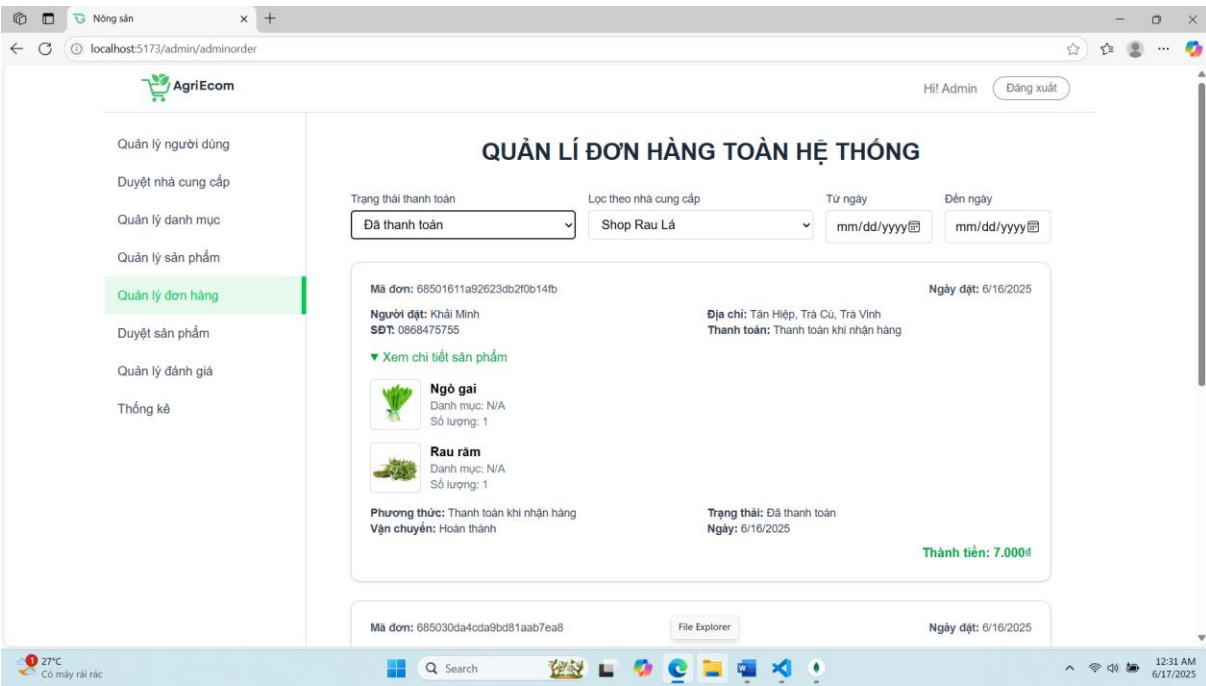
4.3.3. Giao diện quản lý sản phẩm



Hình 4.14 Giao diện quản lý sản phẩm

Mô tả: Giao diện quản lý sản phẩm cho phép quản trị xem tất cả sản phẩm có trên sàn và có thể xem chi tiết sản phẩm và thông tin nhà cung cấp sản phẩm.

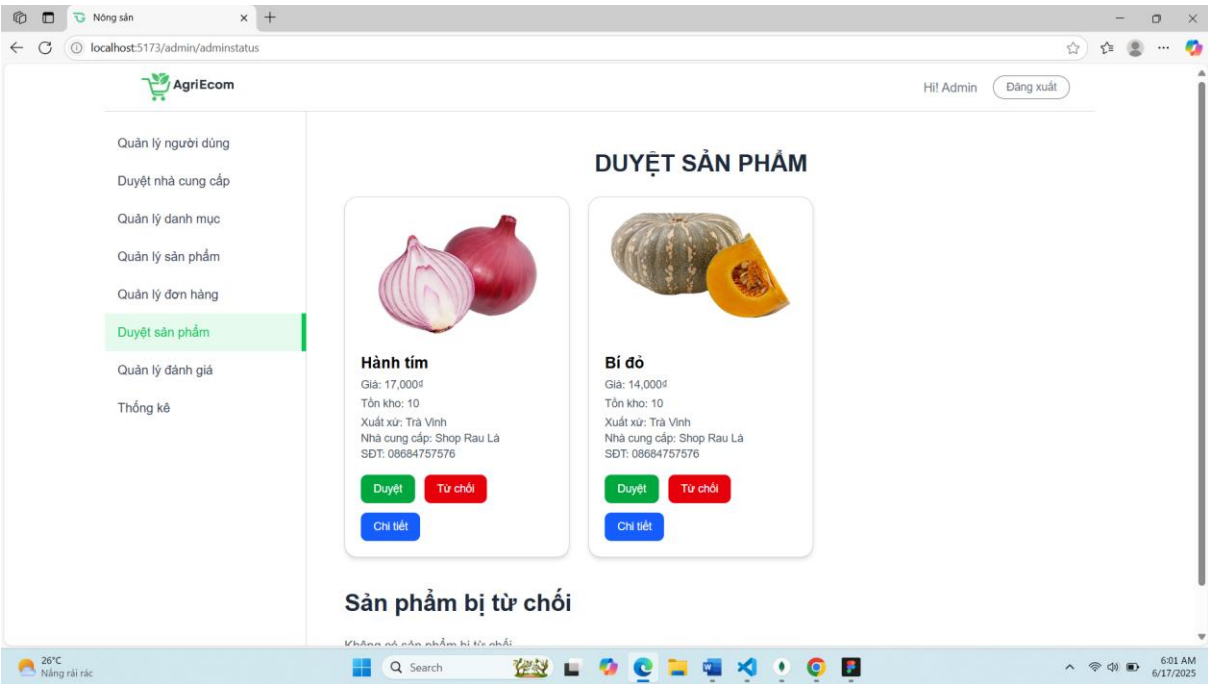
4.3.4. Giao diện quản lý đơn hàng



Hình 4.15 Giao diện quản lý đơn hàng

Mô tả: Quản trị viên có thể xem toàn bộ đơn hàng, có thể lọc theo nhà cung cấp, lọc theo trạng thái thanh toán.

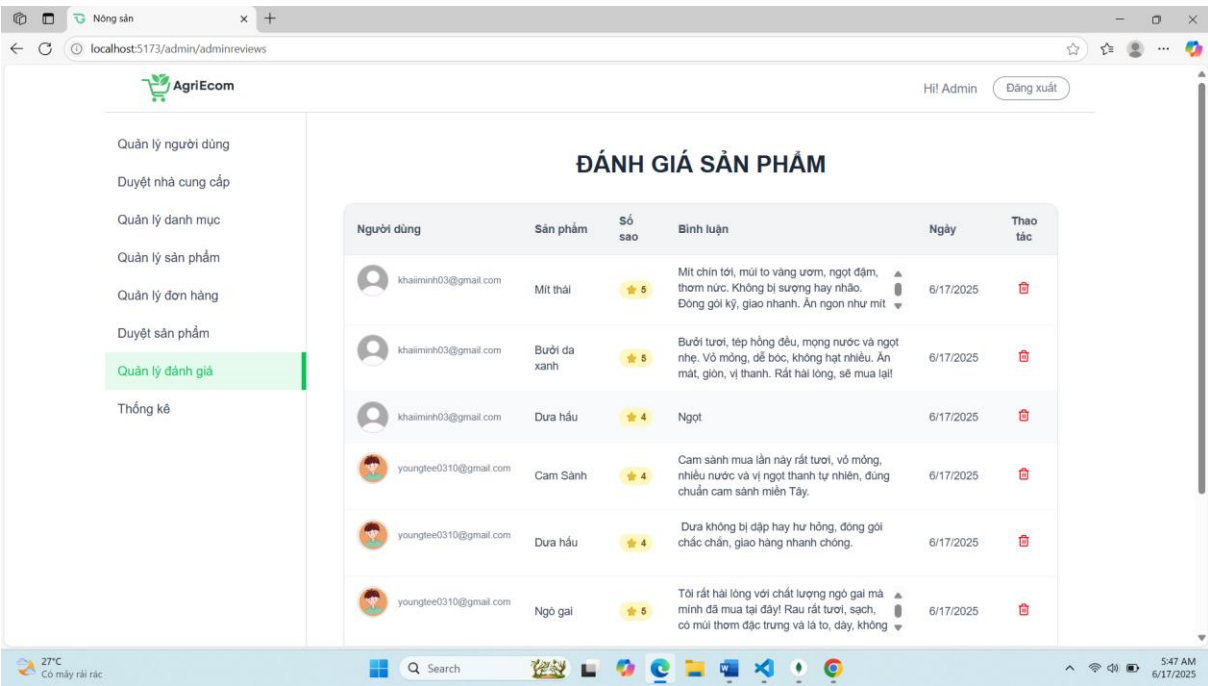
4.3.5. Giao diện duyệt sản phẩm



Hình 4.16 Giao diện duyệt sản phẩm

Mô tả: Giao diện sẽ hiển thị sản phẩm của nhà cung cấp đăng lên khi được duyệt thì mới hiển thị trên sàn.

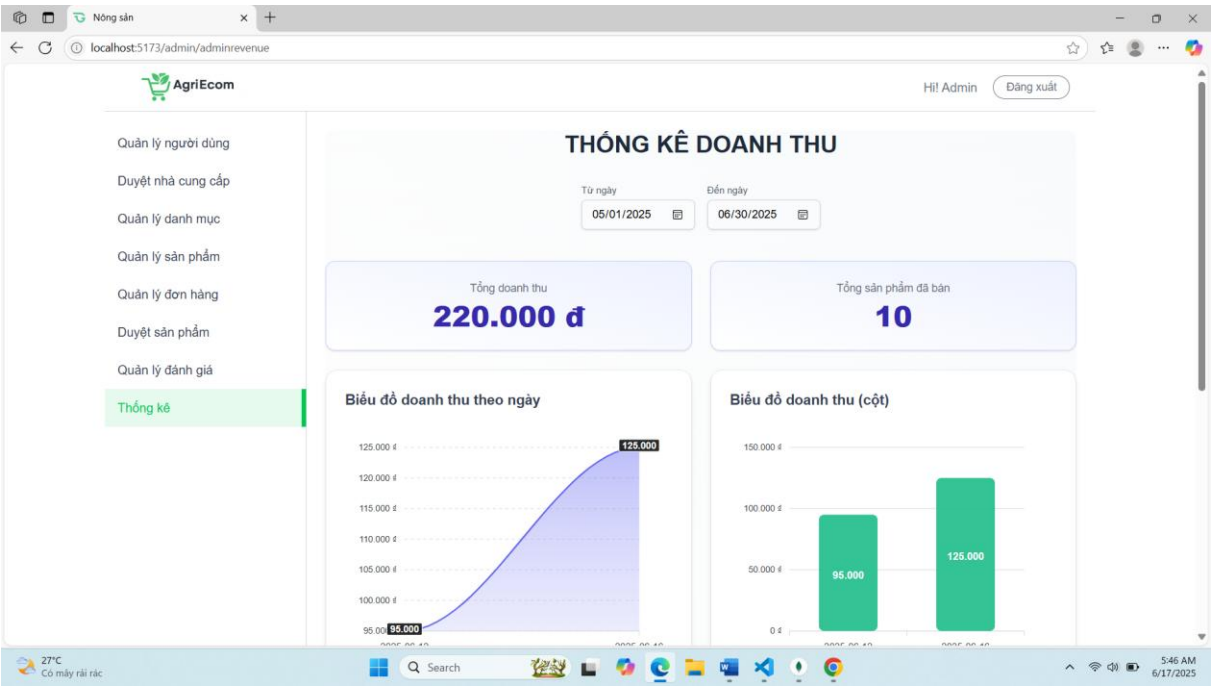
4.3.6. Giao diện đánh giá sản phẩm



Hình 4.17 Giao diện đánh giá sản phẩm

Mô tả: Giao diện này hiển thị tất cả đánh giá của sản phẩm, quản trị có thể xóa bình luận nếu thấy không phù hợp.

4.3.7. Giao diện thống kê doanh thu



Hình 4.18 Giao diện thống kê doanh thu

Mô tả: Thống kê doanh thu theo ngày và các sản phẩm bán chạy, trạng thái đơn hàng, tổng doanh thu, tổng sản phẩm đã bán của toàn bộ hệ thống.

CHƯƠNG: 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong quá trình xây dựng sàn thương mại điện tử chuyên về nông sản, em đã đạt được nhiều kết quả tích cực, đồng thời rút ra những bài học thực tế quan trọng trong việc phát triển phần mềm hiện đại.

Kết quả đạt được:

- Em đã xây dựng thành công một hệ thống web hoàn chỉnh, gồm cả frontend và backend, đáp ứng được đầy đủ các chức năng cốt lõi:
 - Đăng ký, đăng nhập, xác thực người dùng bằng JWT.
 - Chuyển đổi vai trò thành nhà cung cấp, đăng ký hồ sơ và chờ phê duyệt từ quản trị viên.
 - Quản lý sản phẩm, đăng bán, duyệt và tìm kiếm theo danh mục.
 - Tạo đơn hàng, theo dõi trạng thái giao hàng và cập nhật tình trạng đơn.
 - Đánh giá sản phẩm sau khi nhận hàng.
 - Xem báo cáo doanh thu dành cho nhà cung cấp.
 - Giao diện người dùng được xây dựng bằng React, có thiết kế trực quan, hiện đại, tương thích tốt trên nhiều thiết bị.
 - Hệ thống backend sử dụng NestJS, tổ chức mã nguồn rõ ràng theo mô hình module, đảm bảo tính mở rộng và bảo trì dễ dàng.
 - Cơ sở dữ liệu được thiết kế hợp lý trên nền MongoDB, đảm bảo các mối quan hệ giữa các thực thể như người dùng, sản phẩm, đơn hàng, đánh giá, nhà cung cấp,... được liên kết chặt chẽ.

Ưu điểm:

- Áp dụng đúng mô hình kiến trúc client-server, sử dụng chuẩn RESTful API hiệu quả giữa frontend và backend.
- Bảo mật hệ thống tốt với JWT, bảo vệ các route nhạy cảm.
- Giao diện thân thiện với người dùng, đáp ứng các thao tác phổ biến của khách hàng và nhà cung cấp.
- Kiểm thử và xử lý lỗi đầy đủ, đảm bảo tính ổn định khi vận hành.

Hạn chế:

- Chưa có cơ chế liên kết với các đơn vị vận chuyển thật, dẫn đến hạn chế trong việc mô phỏng trải nghiệm giao hàng thực tế.

- Thiếu các tính năng nâng cao như trò chuyện trực tuyến, hệ thống thông báo thời gian thực hoặc theo dõi đơn hàng bằng bản đồ.

5.2. Hướng phát triển

Dù đã đạt được những mục tiêu đề ra ban đầu, nhưng hệ thống vẫn còn nhiều tiềm năng để mở rộng và cải tiến. Trong thời gian tới, em dự định tiếp tục phát triển đề tài theo các hướng sau:

- Phát triển ứng dụng di động: Tạo ra phiên bản mobile giúp người dùng đặt hàng dễ dàng hơn trên điện thoại.
- Tối ưu hóa UI/UX: Nâng cấp giao diện người dùng để phù hợp hơn với đối tượng người dân nông thôn, dễ thao tác và truy cập.
- Mở rộng chức năng quản lý vận chuyển: Cho phép theo dõi tiến trình giao hàng, thông báo trạng thái đơn hàng theo thời gian thực.
- Triển khai thực tế tại địa phương: Kết nối với các hợp tác xã, nông hộ tại Trà Vinh để thử nghiệm triển khai hệ thống trong môi trường thực tế.
- Với định hướng trên em tin rằng hệ thống có thể trở thành một công cụ hữu ích, hỗ trợ bà con nông dân và nhà cung cấp tại địa phương trong việc đưa sản phẩm nông sản đến gần hơn với người tiêu dùng.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] 200LabBlog (2024), *ReactJS là gì? Thư viện Javascript tối ưu cho giao diện Web*, <https://200lab.io/blog/reactjs-la-gi/>. [Truy cập: 12/05/2025].
- [2] AWS (n.d.), *API RESTful là gì?*, <https://aws.amazon.com/vi/what-is/restful-api/>. [Truy cập: 12/05/2025].
- [3] FPT Shop (n.d.), *Tìm hiểu NestJS là gì? Nâng cao kỹ năng lập trình Node.js của bạn với các tính năng tiên tiến*, <https://fptshop.com.vn/tin-tuc/danh-gia/nestjs-la-gi-184643>. [Truy cập: 20/05/2025].
- [4] Stringee (2023), *Node.js là gì? Kiến thức tổng quan từ A-Z về Node.js*, <https://stringee.com/vi/blog/post/nodejs-la-gi#2-Nguyen-ly-hoat-dong-cua-NodeJS>. [Truy cập: 14/05/2025].
- [5] Stringee (2024), *ReactJS là gì? Tất tần tật những điều căn bản về ReactJS*, <https://stringee.com/vi/blog/post/reactJS-la-gi>. [Truy cập: 12/05/2025].
- [6] TopDev (n.d.), *RESTful API là gì? Cách thiết kế RESTful API*, <https://topdev.vn/blog/restful-api-la-gi/#restful-api-la-gi>. [Truy cập: 20/05/2025].
- [7] TopDev (n.d.), *TypeScript là gì? Hướng dẫn cài đặt và sử dụng chi tiết*, <https://topdev.vn/blog/typescript-la-gi/>. [Truy cập: 16/05/2025].
- [8] Viblo (2018), *MongoDB là gì? Cơ sở dữ liệu phi quan hệ*, <https://viblo.asia/p/mongodb-la-gi-co-so-du-lieu-phi-quan-he-bJzKmgoPl9N>. [Truy cập: 12/05/2025].
- [9] Viblo (2018), *Một cái nhìn tổng quan nhất về Nodejs*, <https://viblo.asia/p/mot-cai-nhin-tong-quan-nhat-ve-nodejs-Ljy5VeJ3lra>. [Truy cập: 13/05/2025].

PHỤ LỤC: CÔNG CỤ VÀ CÔNG NGHỆ

1. Công nghệ sử dụng

Frontend:

Framework: React

Ngôn ngữ: TypeScript

Công cụ xây dựng: Vite

Thư viện UI: (Ví dụ) Tailwind CSS, Ant Design, hoặc Material UI (tùy chọn theo dự án)

Giao tiếp backend: Axios / Fetch API (RESTful API)

Backend:

Framework: NestJS

Ngôn ngữ: TypeScript

Cơ sở dữ liệu: MongoDB

ORM / ODM: Mongoose (hoặc Prisma nếu dùng SQL)

Hệ thống xác thực: JWT (JSON Web Token)

API: RESTful API (HTTP)

Cấu trúc dự án: Module-based architecture (theo chuẩn NestJS)

2. Môi trường phát triển

Hệ điều hành: Windows 10 / 11 64-bit (hoặc macOS / Linux tùy máy)

Trình duyệt hỗ trợ: Google Chrome ($\geq v100$), Microsoft Edge ($\geq v100$)

Node.js: v18.x LTS

npm: v9.x hoặc yarn: v1.22.x

MongoDB: v6.x (chạy cục bộ hoặc dùng MongoDB Atlas)

3. Các bước cài đặt ReactJS

Bước 1: Cài đặt Node.js và npm

Trước hết, cần tải và cài đặt phiên bản mới nhất của Node.js

Bước 2: Tạo một ứng dụng React

Sử dụng lệnh sau để tạo một ứng dụng React mới có tên là my-app:

```
npx create-react-app my-app
```

```
cd my-app
```

Sau khi tạo thành công, sẽ có cấu trúc thư mục cơ bản cho ứng dụng React của mình.

Bước 3: Tạo các component

Mở file `src/App.js` và tạo một component đơn giản để hiển thị một "Hello, World!:

```
// src/App.js

import React from 'react';

function App() {

  return (

    <div>

      <h1>Hello, World!</h1>

    </div>

  );

}

export default App;
```

Bước 4: Xây dựng giao diện người dùng

JSX cho phép kết hợp HTML với JavaScript trong React. Thêm các thành phần và giao diện người dùng trong component `App`.

Bước 5: Quản lý trạng thái

Thêm trạng thái vào component bằng cách sử dụng React Hooks (hoặc Class Components nếu muốn):

Bước 6: Kết nối với API

```
// src/App.js

import React, { useState } from 'react';

function App() {

  const [count, setCount] = useState(0);
```

```
return (  
  <div>  
    <h1>Hello, World!</h1>  
    <p>You clicked {count} times.</p>  
    <button onClick={() => setCount(count + 1)}>  
      Click me  
    </button>  
  </div>  
);  
}
```

export default App;

Có thể sử dụng thư viện như Axios hoặc fetch API của JavaScript để gửi các yêu cầu HTTP.

Bước 7: Build và triển khai ứng dụng

Sử dụng lệnh sau để build ứng dụng:

```
npm run build
```

Sau khi build thành công, sẽ có thư mục build chứa các file tĩnh của ứng dụng. Khởi động dự án bằng lệnh *npm run dev*

Cài đặt NestJs

Để cài đặt NestJS, có thể làm theo các bước sau:.

Mở Terminal hoặc Command Prompt và chạy lệnh sau để cài đặt NestJS CLI (Command Line Interface):

```
npm install -g @nestjs/cli
```

Sau khi cài đặt thành công, có thể tạo một dự án NestJS mới bằng cách chạy lệnh sau:

```
nest new project-name
```

Trong đó, "project-name" thể hiện tên dự án. NestJS sẽ tạo ra một thư mục mới với tên dự án và cài đặt các thành phần cần thiết.

Di chuyển vào thư mục dự án mới:

```
cd project-name
```

Cuối cùng, có thể chạy dự án NestJS bằng lệnh:

```
npm run start
```

Nếu muốn khởi động lại dự án mỗi khi thay đổi mã nguồn, có thể sử dụng lệnh:

```
npm run start:dev
```